

Automati, Calcolabilità e Complessità

Alessio Marini, 2122855

Appunti presi durante il corso di **Automati, Calcolabilità e Complessità** nell'anno **2025/2026** del professore Daniele Venturi.

Gli appunti li scrivo principalmente per rendere il corso più comprensibile **a me** e anche per imparare il linguaggio Typst. Se li usate per studiare verificate sempre le informazioni 🙏.

Contatti:

📧 alem1105

✉ marini.2122855@studenti.uniroma1.it

September 27, 2025

Contents

1. Automi	3
1.1. Linguaggi Regolari	3
1.2. Operazioni sui Linguaggi	7
1.3. Non Determinismo	7
1.4. Equivalenza tra DFA e NFA	9
1.5. Chiusura per Linguaggi Regolari	12
1.6. Espressioni Regolari	16
1.7. Pumping Lemma	21
1.8. Linguaggi Acontestuali	23
1.8.1. Unione tra CFG	25
1.8.2. Trasformare un DFA in un CFG	25
1.8.3. Forma Normale di Chomsky	26
1.9. Pushdown Automata - PDA	28
1.10. Pumping Lemma per i CFL	33
2. Calcolabilità	35
2.1. TM Multinastro	37
2.2. TM non Deterministica	37
2.3. Linguaggi Decidibili	38
2.4. Riducibilità	43
2.5. Halting Problem	44
2.6. Altri linguaggi non decidibili	44
2.7. I Teoremi di Gödel	46
2.7.1. Primo Teorema di Incompletezza	47
2.7.2. Secondo Teorema di Incompletezza	48
3. Complessità	48
3.1. Satisfiability	50
3.2. Space Complexity	59

1. Automi

1.1. Linguaggi Regolari

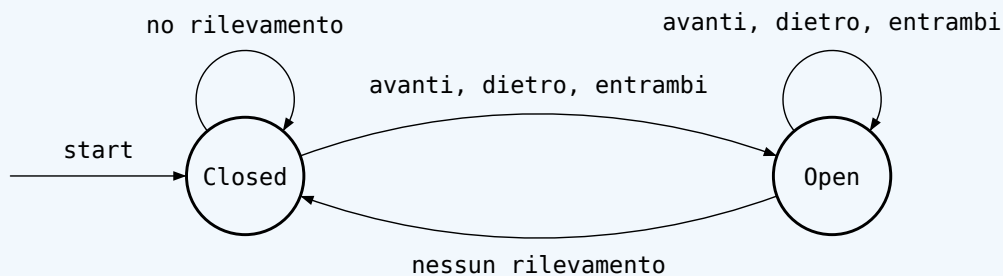
Iniziamo introducendo in modo *informale* cos'è un **Automa**.

Un Automa è un modello matematico di calcolo che può elaborare informazioni e agire in modo automatico seguendo una serie di stati predefiniti.

Esistono diversi tipi di automi, il primo che vediamo si chiama **Automa a stati Finiti Deterministico (DFA)**, questo tipo di automa ha un **numero finito di stati** e processa gli input che riceve in modo **sequenziale** bit a bit.

Esempio - Sistema di controllo di una porta automatica

La porta ha dei sensori “avanti” e “dietro” per rilevare i movimenti ed aprire la porta.



Adesso però diamo una definizione più formale per i DFA.

Definizione - DFA

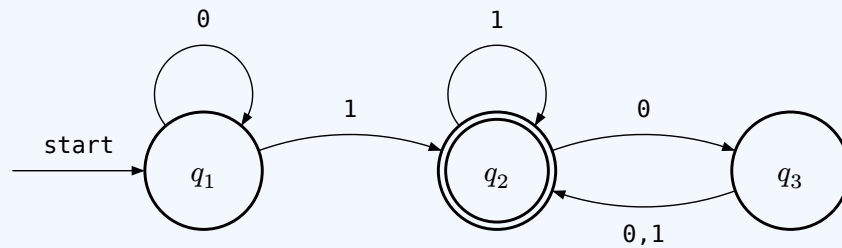
Un DFA è una quintupla $(Q, \Sigma, \delta, q_0, F)$ dove:

- Q : Insieme finito degli stati
- Σ : Insieme finito dei simboli in input (**alfabeto**)
- $\delta : Q \times \Sigma \rightarrow Q$: **Funzione di Transizione**, ci dice come avvengono i passaggi di stato
- q_0 : Stato **iniziale**
- $F \subseteq Q$: Insieme degli **stati di accettazione**

Nei nostri diagrammi indicheremo gli stati di accettazione, o finali, con un *doppio bordo*. Inoltre diciamo che un automa “accetta” un input se termina in uno stato di accettazione, altrimenti diremo che lo “rifiuta”.

Vediamo quindi un altro esempio di DFA.

Esempio - DFA di un linguaggio binario



Vediamo i vari elementi rispetto alla definizione che abbiamo dato prima:

- $Q = \{q_1, q_2, q_3\}$
- $\Sigma = \{0, 1\}$
- $q_0 = q_1$
- $F = \{q_2\}$

Mentre per la δ possiamo o rappresentare una tabella o in questo caso semplicemente guardare le frecce del diagramma. Questo DFA, ad esempio, *accetta* se riceve come input la stringa $w=11101$

Introduciamo altre definizioni che ci serviranno per definire in modo formale il concetto di **linguaggio accettato**.

In modo informale un linguaggio è l'insieme di tutte le stringhe riconosciute da un DFA, se ad esempio abbiamo il DFA M allora l'insieme delle stringhe riconosciute da M lo denotiamo con $L(M)$. Ad esempio il DFA dell'esempio precedente ha come linguaggio:

$$A = \{w \mid w \text{ contiene almeno un '1' e un numero pari di '0' segue l'ultimo '1'}\}$$

E possiamo dire quindi che $L(M) = A$ o anche “ M riconosce A ”.

Per definire il linguaggio accettato dobbiamo modificare la funzione di transizione e introdurre la **funzione di transizione estesa**.

Definizione - Funzione di Transizione Estesa

La denotiamo con δ^* e ha lo stesso funzionamento della funzione di transizione classica, la differenza è che insieme allo stato accetta un'intera stringa e non un solo carattere.

- $\delta^* : Q \times \Sigma^* \rightarrow Q$
- $\delta^*(q, \varepsilon) = q$
- $\delta^*(q, ax) = \delta^*(\delta(q, a), x)$

Con Σ^* indichiamo l'insieme delle stringhe in input e $x \in \Sigma^*, a \in \Sigma$. Questo significa quindi che la funzione di transizione estesa processa intere stringhe ma sempre in modo sequenziale carattere per carattere.

Introduciamo anche il concetto di **configurazione**, ovvero lo stato in cui si trova il DFA, in particolare è definita come una tupla $(q, x) \in Q \times \Sigma^*$ e indica appunto lo stato in cui si trova l'automa e quello che gli resta da leggere. La configurazione iniziale è quella che si trova nello stato q_0 quindi ad esempio (q_0, x) .

Adesso possiamo definire anche il **passo di computazione** ovvero una relazione binaria tra configurazioni che ci porta appunto da una configurazione ad un'altra. Si ha:

$$(p, ax) \vdash_M (q, x) \Leftrightarrow \delta(p, a) = q$$

Con $p, q \in Q, a \in \Sigma, x \in \Sigma^*$. Stiamo dicendo quindi che passiamo dalla configurazione (p, ax) alla configurazione (q, x) leggendo a (ci resta poi da leggere x) che è equivalente a dire che da p passiamo a q se leggiamo a .

Anche la relazione binaria $\vdash_M$ può essere estesa considerando la **chiusura riflessiva e transitiva** e la denotiamo con $\vdash_M^*$:

- $(q, x) \vdash_M^* (q, x)$, ovvero quando non legge nessun input.
- $(q, aby) \vdash_M (p, by) \wedge (p, by) \vdash_M (r, y) \Rightarrow (q, aby) \vdash_M^* (r, y)$, possiamo quindi raccogliere più passi di configurazione.

Con $p, q, r \in Q; a, b \in \Sigma; y \in \Sigma^*$

Adesso possiamo finalmente utilizzare tutti questi concetti per formalizzare la definizione di **linguaggio accettato**.

Definizione - Linguaggio Accettato

Diciamo che $x \in \Sigma^*$ è accettato da $M = (Q, \Sigma, \delta, q_0, F)$ se

$$\delta^*(q_0, x) \in F \quad \text{oppure} \quad (q_0, x) \vdash_M^* (q, \varepsilon) \text{ con } q \in F$$

Possiamo definirlo anche come insieme ovvero $L(M) = \{x \in \Sigma^* : \delta^*(q_0, x) \in F\}$

Adesso che sappiamo cos'è un DFA e il suo linguaggio accettato, possiamo definire i **linguaggi regolari**.

Definizione - Linguaggio Regolare

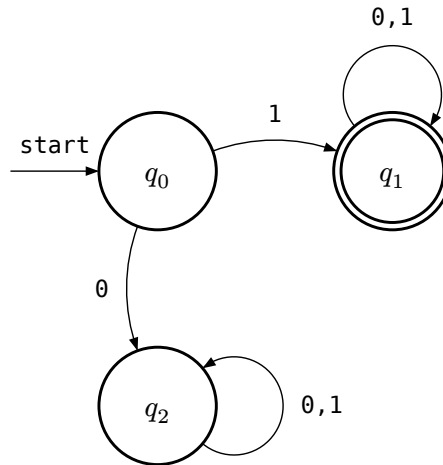
Un linguaggio è chiamato **regolare** se un DFA lo riconosce, indichiamo con REG l'insieme dei linguaggi regolari:

$$\text{REG} = \{L \subseteq \Sigma^* \mid \exists \text{ DFA } M \text{ t.c. } L(M) = L\}$$

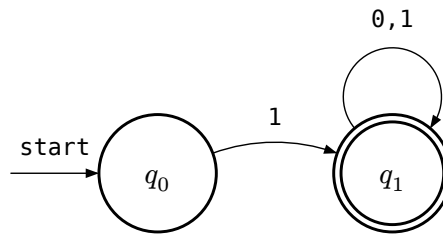
Ci interessa capire come costruire un DFA dato un certo linguaggio, ad esempio prendiamo il linguaggio:

$$L = \{x \in \{0, 1\}^* \mid x = 1\|y, y \in \{0, 1\}^*\}$$

L'operatore $\|$ indica la concatenazione di stringhe, in questo caso quindi stiamo indicando tutte le stringhe che iniziano con 1. Proviamo a costruire un DFA A che riconosce il linguaggio:



Lo stato q_2 viene chiamato “stato pozzo”, ovvero uno stato non di accettazione che non ha archi uscenti verso altri stati. Possiamo disegnare anche un DFA equivalente senza gli archi di q_2 :



In questo caso se in q_0 riceviamo 0 semplicemente ignoriamo l’input.

Adesso però vogliamo dimostrare la correttezza del DFA appena costruito, ovvero dimostrare che il DFA accetta x **se e solo se** $x \in L$. Costruiamo una dimostrazione per induzione, per prima cosa però notiamo due cose nel nostro DFA:

- $\delta^*(q_1, u) = q_1 \quad \forall u \in \{0, 1\}^*$
- $\delta^*(q_2, u) = q_2 \quad \forall u \in \{0, 1\}^*$

Stiamo dicendo che se entriamo in q_1 o q_2 non andremo più in altri stati. Adesso iniziamo la dimostrazione per induzione:

- **Caso Base** - Abbiamo $|x| = 0$ ovvero $x = \varepsilon$ stringa vuota. Si ha che

$$\delta^*(q_0, \varepsilon) = \delta(q_0, \varepsilon) = q_0 \notin F$$

- **Ipotesi Induttiva** - Sia $n > 0$, supponiamo $|w| \leq n$. Si ha:

$$\delta^*(q_0, w) = \begin{cases} q_1 & \text{se } w \text{ inizia con } 1 \\ q_2 & \text{se } w \text{ inizia con } 0 \end{cases}$$

Possiamo dirlo perchè abbiamo visto all’inizio che una volta entrati in q_1 o q_2 il DFA non cambia più stato.

- **Passo Induttivo** - Prendiamo x t.c. $|x| = n + 1$, possiamo scomporla in $x = au$ con $a \in \{0, 1\}$ e $u \in \{0, 1\}^*$. Otteniamo quindi

$$\delta^*(q_0, x) = \delta^*(q_0, au) = \delta^* \left(\underbrace{\delta(q_0, a)}_{\text{ha 2 soluzioni}}, u \right)$$

Le 2 soluzioni sono quelle che abbiamo visto nell’ipotesi induttiva, infatti:

$$\delta(q_0, a) = \begin{cases} q_2 & \text{se } a = 0 \\ q_1 & \text{se } a = 1 \end{cases}$$

Dato che abbiamo dimostrato che l'automa non uscirà mai da nessuno dei due stati e che entra sicuramente in uno dei due possiamo quindi dire che:

$$\delta^*(q_0, x) = q_1 \Leftrightarrow a = 1$$

1.2. Operazioni sui Linguaggi

I linguaggi non sono altro che insiemi di stringhe, questo significa che possiamo applicare delle operazioni su di essi, vediamo alcune:

- **Unione:** $L_1 \cup L_2 = \{x \in \Sigma^* \mid x \in L_1 \vee x \in L_2\}$
- **Intersezione:** $L_1 \cap L_2 = \{x \in \Sigma^* \mid x \in L_1 \wedge x \in L_2\}$
- **Complemento:** $\overline{L_1} = \{x \in \Sigma^* \mid x \notin L_1\}$
- **Concatenazione:** $L_1 \circ L_2 = \{xy \mid x \in L_1 \wedge y \in L_2\}$ importante notare che non è commutativi quindi $L_1 \circ L_2 \neq L_2 \circ L_1$.

Vediamo un esempio, prendiamo:

- $\Sigma = \{a, b\}$
- $L_1 = \{a, ab, ba\}$
- $L_2 = \{ab, b\}$

Otteniamo $L_1 \circ L_2 = \{aab, ab, abab, abb, baab, bab\}$

- **Potenza:** La definiamo in modo ricorsivo

$$\text{per stringhe } \begin{cases} x^0 = \varepsilon \\ x^{n+1} = x^n x \end{cases} ; \text{ per linguaggi } \begin{cases} L^0 = \{\varepsilon\} \\ L^{n+1} = L^n \circ L \end{cases}$$

Quindi ad esempio se prendiamo $L = \{a, ab, ba\}$ possiamo dire:

$$L^2 = L \circ L = \{aa, aab, aba, aba, abab, abba, baa, baab, baba\}$$

- **Operatore *:** $L^* = \bigcup_{n \geq 0} L^n = \{\varepsilon\} \cup L^1 \cup L^2 \cup \dots$ ovvero tutte le combinazioni di stringhe possibili. Ad esempio se $L = \{a, b\}$ abbiamo $L^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$

1.3. Non Determinismo

Per ora abbiamo visto come un DFA che si trova in uno stato quando processa un input farà un solo movimento verso un altro stato, oppure rimane nello stesso. Nel non determinismo invece abbiamo dei comportamenti diversi:

- Quando un automa si trova in uno stato e processa un input può andare in **diversi** stati **contemporaneamente**.
- Sono ammessi gli ε -archi, ovvero degli archi che non hanno bisogno di input per essere percorsi.
- L'automa a questo punto, visto che esplora più rami di computazione contemporaneamente, accetta se esiste almeno un ramo che accetta. Vedremo successivamente cosa significa "ramo di computazione".

A questo punto non parliamo più di DFA ma di **NFA** ovvero automi **non deterministici** a stati finiti.

Definizione - NFA

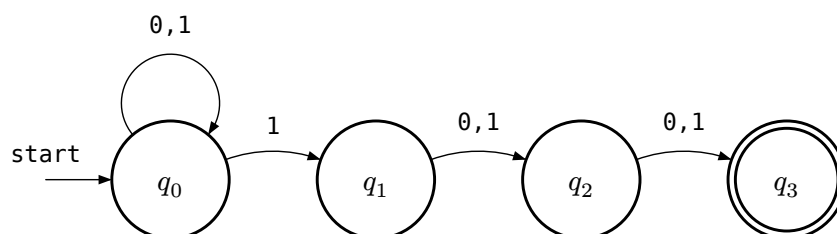
Un NFA è una tupla $(Q, \Sigma, \delta, q_0, F)$ dove tutti gli elementi sono come per i DFA ma abbiamo un cambio nella funzione di transizione. infatti questa diventa:

- $\delta : Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q)$ con Σ_ϵ indichiamo $\Sigma \cup \{\epsilon\}$, mentre con $\mathcal{P}(Q)$ l'insieme delle parti di Q ovvero l'insieme di tutti i possibili sottoinsiemi di Q

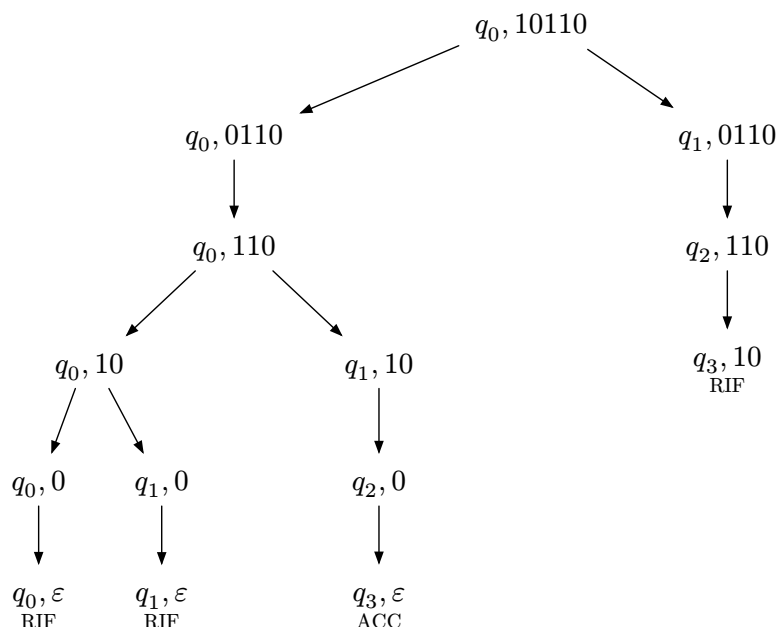
Per chiarire il concetto di “ramo di computazione” vediamo un esempio di NFA. Costruiamo un NFA che riconosce il linguaggio

$$L = \{x : x \in \{0,1\}^* \wedge x \text{ ha un '1' in terzultima posizione}\}$$

Costruiamo l'NFA:



Cosa succede quando l'NFA computa la stringa 10110? In un NFA quando uno stato permette più transizioni si creano più rami, uno per ogni strada presa, ad esempio con l'NFA appena costruito se inseriamo la stringa 10110 con il primo carattere 1 l'NFA andrà sia in q_0 che in q_1 e questo porta la creazione di due rami di computazione. L'automa accetta se esiste almeno un ramo fra questi che accetta. Per chiarezza vediamo quali rami si creano in questo esempio:

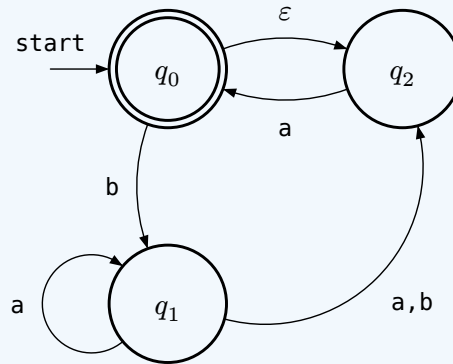


Per quanto riguarda gli ϵ -archi invece, se l'automa si trova in uno stato che ne possiede uno allora l'automa si duplica in due rami senza leggere nessun input:

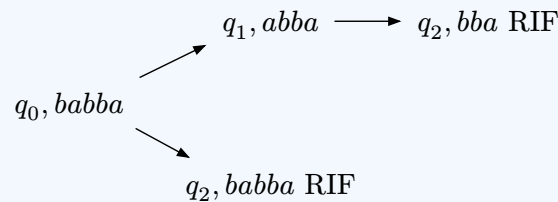
- Un ramo rimane nella stessa configurazione di prima.
- L'altro ramo segue l' ϵ -arco.

Vediamo un altro esempio per chiarire anche questo concetto.

Esempio - NFA con ε -archi



Vediamo cosa succede con la computazione della stringa babba



Dobbiamo adesso rivedere i concetti di **configurazione, computazione ed accettazione** in un NFA:

- Dato un NFA N , indichiamo come configurazione una coppia $(q, x) \in Q, \Sigma_\varepsilon^*$
- Avremo

$$(p, ax) \vdash_N (q, x) \Leftrightarrow q \in \delta(p, a) \quad \text{con } x \in \Sigma_\varepsilon^*; a \in \Sigma_\varepsilon; p, q \in Q$$

Da notare che q, p non sono più degli stati singoli ma un insieme di stati infatti $q, p \in \mathcal{P}(Q)$, ovviamente possono anche contenere un singolo stato ma saranno comunque insiemi di stati.

- Un NFA N accetta

$$w \in \Sigma_\varepsilon^* \Leftrightarrow \exists q \in F \text{ t.c. } (q_0, w) \vdash_N^* (q, \varepsilon)$$

1.4. Equivalenza tra DFA e NFA

Dimostriamo come in realtà le due classi di automi sono equivalenti. Per prima cosa definiamo in modo formale le due classi:

- $\mathcal{L}(\text{DFA}) = \text{REG}$, quindi la classe dei linguaggi regolari, riconosciuti da almeno un DFA.
- $\mathcal{L}(\text{NFA}) = \{L \mid \exists \text{ NFA } N \text{ t.c. } L(N) = L\}$, la classe dei linguaggi riconosciuti da almeno un NFA.

Teorema

$$\text{REG} := \mathcal{L}(\text{DFA}) = \mathcal{L}(\text{NFA})$$

Dimostrazione - Formalmente vogliamo dimostrare $\mathcal{L}(\text{DFA}) \subseteq \mathcal{L}(\text{NFA})$ e $\mathcal{L}(\text{NFA}) \subseteq \mathcal{L}(\text{DFA})$

1. Prima implicazione $\mathcal{L}(\text{DFA}) \subseteq \mathcal{L}(\text{NFA})$

Dato $L \in \mathcal{L}(\text{DFA})$ e $D := (Q, \Sigma, \delta, q_0, F)$ t.c. $L = L(D)$. Notiamo che il concetto di NFA, in realtà, non è altro che una generalizzazione di un DFA, infatti se prendiamo un qualsiasi DFA possiamo dire che è anche un NFA, quindi D è un NFA e $L \in \mathcal{L}(\text{NFA})$.

2. Seconda implicazione $\mathcal{L}(\text{NFA}) \subseteq \mathcal{L}(\text{DFA})$

Per dimostrare questa implicazione prendiamo un NFA e costruiamo un DFA che lo simula. Sia

- $N = \{Q, \Sigma, \delta, q_0, F\}$ l'NFA t.c. $L(N) = A$

costruiamo il DFA

- $M = \{Q_M, \Sigma, \delta_M, q_0^M, F_M\}$ t.c. $L(M) = A$

Prima facciamo una costruzione senza considerare gli ε -archi:

- $Q_M = \mathcal{P}(Q)$ - Ogni stato di M è un insieme di stati N
- $\delta_M(R, a) = \bigcup_{r \in R} \delta(r, a)$

Dove $R \in Q_M, a \in \Sigma, \delta(R, a) = \{q \in Q \mid q \in \delta(r, a) \text{ con } r \in R\}$, stiamo dicendo che la funzione di transizione si applica su un insieme di stati e restituisce un altro insieme di stati, precisamente quelli che si ottengono applicando la funzione di transizione su tutti gli elementi dell'insieme in input.

- $q_0^M = \{q_0\}$
- $F_M = \{R \in Q_M \mid R \cap F \neq \emptyset\}$ ovvero quegli insiemi di stati che hanno al loro interno almeno uno stato accettante del DFA.

A questo punto introduciamo gli ε -archi e per farlo definiamo

$$E(R) = \{q \mid q \text{ può essere raggiunto da } R \text{ attraverso 0 o più } \varepsilon\text{-archi}\}$$

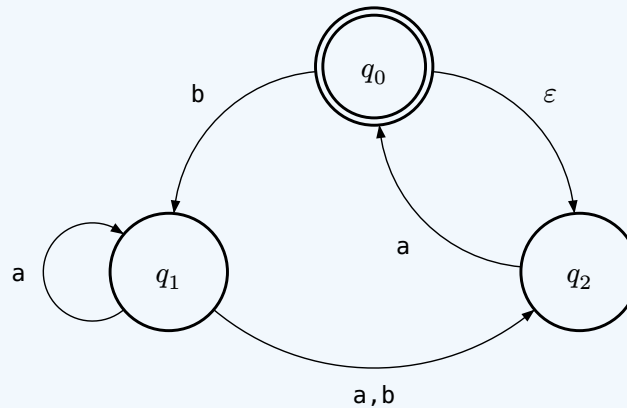
Scriviamo 0 o più perchè in questo modo includiamo anche il nodo su cui ci troviamo. Con questa definizione possiamo modificare la funzione di transizione del DFA e lo stato iniziale del DFA in:

- $\delta_M(R, a) = \{q \in Q \mid q \in E(\delta(r, a)), \exists r \in R\}$. In questo modo includiamo sia gli archi visti con la definizione di prima ma anche quelli che raggiungiamo tramite gli ε -archi.
- $q_0^M = \{q_0\}$

Vediamo un esempio di trasformazione da NFA a DFA per chiarire i concetti.

Esempio - Da NFA a DFA

Prendiamo questo NFA come esempio, ha come linguaggio $\Sigma = \{a, b\}$



Iniziamo a dare le prime definizioni:

- $Q_D = \{q_{\emptyset}, q_{\{0\}}, q_{\{1\}}, q_{\{2\}}, q_{\{0,1\}}, q_{\{0,2\}}, q_{\{1,2\}}, q_{\{0,1,2\}}\}$. Con la notazione $q_{\{0,1\}}$ indichiamo l'insieme di stati $\{q_0, q_1\}$
- $q_0^D = E(q_{\{0\}}) = q_{\{0,2\}}$. Infatti da q_0 ci muoviamo con un ε -arco in q_2 ma manteniamo anche il ramo che si trova in q_0 .
- $F_D = \{q_{\{0\}}, q_{\{0,1\}}, q_{\{0,2\}}, q_{\{0,1,2\}}\}$. Ovvero tutti gli stati che contengono lo stato q_0

Adesso ci manca da definire la funzione di transizione, è un passaggio lungo quindi vediamo solo alcuni casi.

Per lo stato $q_{\{0\}}$ abbiamo:

- $\delta_D(q_{\{0\}}, a) = E(\delta(q_0, a)) = q_{\{0,2\}}$
- $\delta_D(q_{\{0\}}, b) = E(\delta(q_0, b)) = q_{\{1,2\}}$

Per lo stato $q_{\{1\}}$:

- $\delta_D(q_{\{1\}}, a) = E(\delta(q_1, a)) = q_{\{1,2\}}$
- $\delta_D(q_{\{1\}}, b) = E(\delta(q_1, b)) = q_{\{2\}}$

...

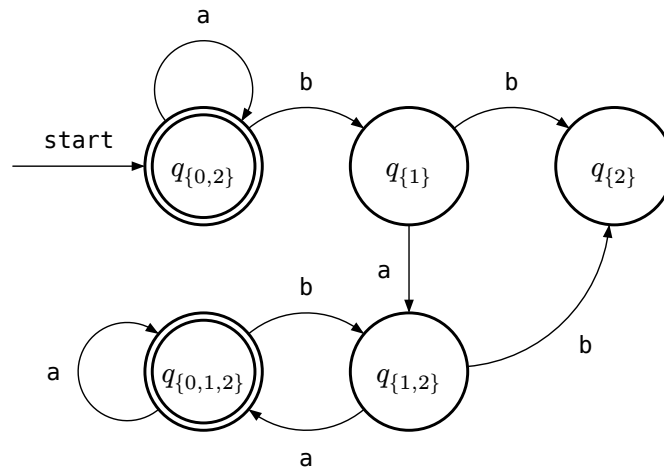
Per lo stato $q_{\{0,1\}}$:

- $\delta_D(q_{\{0,1\}}, a) = E(\delta(q_0, a)) \cup E(\delta(q_1, a)) = q_{\{0,2\}} \cup q_{\{1,2\}} = q_{\{0,1,2\}}$

• ...

...

Basandoci sull'esempio di prima possiamo anche provare a costruire il DFA che abbiamo soltanto definito. (semplificando anche alcuni nodi mai raggiunti)



Da adesso in poi quindi non ci importa più distinguere NFA e DFA dato che abbiamo visto essere equivalenti, possiamo usare benissimo entrambi in qualsiasi situazione, ovviamente in alcuni casi potrebbe essere più vantaggioso un determinato tipo.

1.5. Chiusura per Linguaggi Regolari

I linguaggi regolari sono **chiusi** rispetto alle loro operazione? Ovvero, se applichiamo un'operazione a due linguaggi regolari otteniamo come risultato un altro linguaggio regolare?

Teorema

L'insieme REG è chiuso per unione. Ovvero $L_1, L_2 \in \text{REG} \Rightarrow L_1 \cup L_2 \in \text{REG}$

Dimostrazione - Possiamo svolgere la dimostrazione in due modi:

1. Senza NFA

Per dimostrare il teorema utilizzando i DFA dobbiamo costruire un automa M che riconosce il linguaggio $L_1 \cup L_2$. Per farlo dobbiamo fare in modo che M simuli il comportamento di altri due automi M_1, M_2 tali che $L(M_1) = L_1$ e $L(M_2) = L_2$, e che accetti soltanto se almeno una delle due simulazioni accetta.

Dati

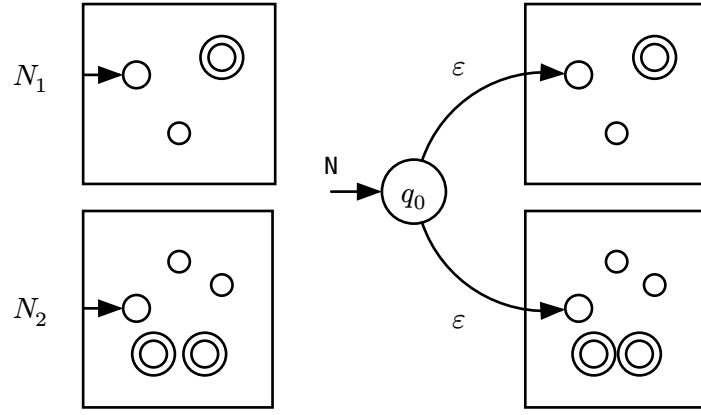
- $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ t.c. $L(M_1) = L_1$
- $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ t.c. $L(M_2) = L_2$

Costruiamo quindi $M = (Q, \Sigma, \delta, q_0, F)$ dove:

- $Q = Q_1 \times Q_2 = \{(r_1, r_2) \mid r_1 \in Q_1 \wedge r_2 \in Q_2\}$
- $\delta : Q \times \Sigma \rightarrow Q$. Ad esempio $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- $q_0 = (q_1, q_2)$
- $F = \{(r_1, r_2) \mid r_1 \in F_1 \vee r_2 \in F_2\} = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

2. Con NFA

In questo caso la dimostrazione è molto semplice, infatti ci basiamo sempre sull'idea di prima di usare due simulazioni N_1, N_2 per i due linguaggi ma le inglobiamo in un NFA N che avrà come nodo iniziale un nuovo nodo collegato ai nodi iniziali delle simulazioni tramite ϵ -archi:



Più formalmente, dati

- $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ t.c. $L(N_1) = L_1$
- $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ t.c. $L(N_2) = L_2$

Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ dove:

- $Q = \{q_0\} \cup Q_1 \cup Q_2$
- $F = F_1 \cup F_2$
- $\delta : Q \times \Sigma \rightarrow Q$ dove

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0, a = \varepsilon \\ \emptyset & q = q_0, a \neq \varepsilon \end{cases}$$

Teorema

REG è chiuso per complemento. Ovvero $\forall L \in \text{REG}, \bar{L} \in \text{REG}$

Dimostrazione - Costruisco un automa $\bar{L}$ che accetta tutto tranne le stringhe accettate da L . Per farlo ci basta avere come stati finali il complemento dell'insieme degli stati finali.

Dato $L = (Q, \Sigma, \delta, q_0, F)$ t.c. $L(L) = A$ costruisco $\bar{L} = (Q, \Sigma, \delta, q_0, Q - F)$

Teorema

REG è chiuso per intersezione. Ovvero $\forall L_1, L_2 \in \text{REG}, L_1 \cap L_2 \in \text{REG}$

Dimostrazione - Dati $D_1 = (Q, \Sigma, \delta_1, q_1, F_1)$ t.c. $\mathcal{L}(D_1) = L_1$ e $D_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ t.c. $\mathcal{L}(D_2) = L_2$ costruisco $M = (Q, \Sigma, \delta, q_0, F)$ t.c.:

- $Q = Q_1 \times Q_2$
- $F = F_1 \times F_2$
- $q_0 = (q_1, q_2)$
- $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$

Teorema

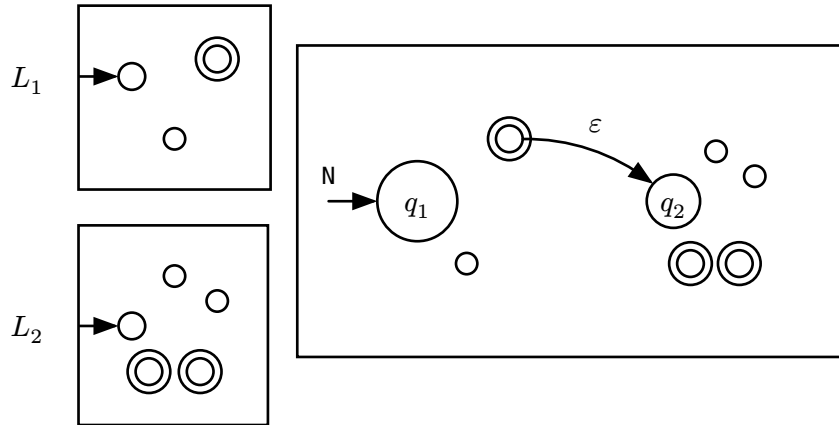
REG è chiuso per concatenazione. $\forall L_1, L_2 \in \text{REG}, L_1 \circ L_2 \in \text{REG}$

Dati $D_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ t.c. $\mathcal{L}(D_1) = L_1$ e $D_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ t.c. $\mathcal{L}(D_2) = L_2$ costruisco $N = (Q, \Sigma, \delta, q_0, F)$ t.c.:

- $Q = Q_1 \cup Q_2$
- $q_0 = q_1$
- $F = F_2$
- $\forall q \in Q, a \in \Sigma$ si ha che

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 - F_1 \\ \delta_1(q, a) & q \in F_1 \wedge a \neq \varepsilon \\ \delta_1(q, a) \cup \{q_2\} & q \in F_1 \wedge a = \varepsilon \\ \delta_2(q, a) & q \in Q_2 \end{cases}$$

Graficamente ci basta prendere il primo automa L_1 e collegare i suoi stati finali allo stato iniziale di L_2 tramite ε -archi:



Da questa dimostrazione possiamo anche dire che REG è chiuso per potenza, dato che la potenza si esprime in funzione della concatenazione. Es $L^2 = L \circ L$

Corollario

REG è chiuso per l'operatore potenza. $\forall L \in \text{REG}, n \in \mathbb{N}, L^n \in \text{REG}$.

Teorema

REG è chiuso per l'operatore $*$. $\forall L \in \text{REG}, L^* \in \text{REG}$.

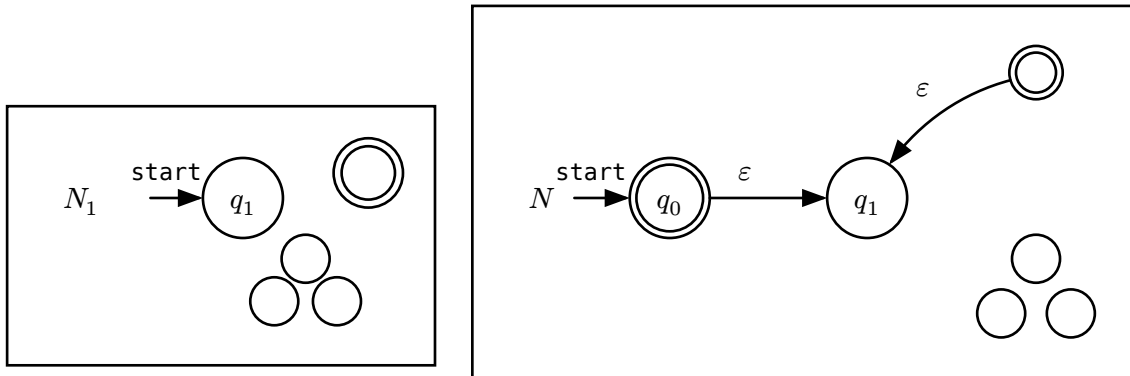
Dimostrazione - Dato $L \in \text{REG}$, sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F)$ l'NFA t.c. $\mathcal{L}(N_1) = L$ costruisco $N = (Q, \Sigma, \delta, q_0, F)$ NFA t.c.:

- $q_0 \neq q_1$ è un nuovo stato iniziale
- $Q = Q_1 \cup \{q_0\}$
- $F = F_1 \cup \{q_0\}$. Dobbiamo aggiungere anche il nuovo stato iniziale dato che l'operatore $*$ comprende anche la stringa vuota ε .

- $\forall q \in Q, a \in \Sigma$ si ha che

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q - F \\ \delta_1(q, a) & q \in F \wedge a \neq \varepsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F \wedge a = \varepsilon \\ \{q_1\} & q = q_0 \wedge a = \varepsilon \\ \emptyset & q = q_0 \wedge a \neq \varepsilon \end{cases}$$

Questo nuovo automa accetta tutto quello che accetta N_1 ma ogni volta che accetta torna indietro con un ε -arco per provare ad accettare un'altra stringa accettata sempre da N_1 . Proviamo a rappresentare questo automa:



Come prima, grazie a questa dimostrazione possiamo anche dire che REG è chiuso per $+$. L'operatore $+$ è come l'operatore $*$ ma esclude la potenza 0, possiamo definirlo quindi come $L^+ = L^1 \cup L^2 \cup \dots$

Notazione

L^+ equivale quindi a scrivere LL^* , ovvero almeno una concatenazione di L .

Corollario

REG è chiuso per $+$. La dimostrazione è uguale a $*$ ma ci basta escludere q_0 da F .

1.6. Espressioni Regolari

Le espressioni regolari sono come le espressioni matematiche ma invece di rappresentare numeri, rappresentano linguaggi. Diamo una definizione formale.

Definizione - Espressione Regolare

Sia Σ un alfabeto. Un'espressione regolare su Σ che indichiamo con $re(\Sigma)$ è definita ricorsivamente come segue:

$$\begin{aligned} \text{caso base: } & \begin{cases} \emptyset & \in re(\Sigma) \\ \varepsilon & \in re(\Sigma) \\ a & \in re(\Sigma), \forall a \in \Sigma \end{cases} \\ \text{caso induttivo: } & \begin{cases} R_1 \cup R_2 & R_1, R_2 \in re(\Sigma) \\ R_1 \circ R_2 & R_1, R_2 \in re(\Sigma) \\ (R_1)^* & R_1 \in re(\Sigma) \end{cases} \end{aligned}$$

Dove R_i sono espressioni regolari.

Stiamo quindi dicendo che, preso un alfabeto, i suoi singoli caratteri, la stringa vuota e l'insieme vuoto appartengono all'espressione regolare che descrive questo alfabeto. L'unione, l'intersezione o la * di espressioni regolari su questo alfabeto sono comunque espressioni regolari dell'alfabeto.

Osservazione - Ogni espressione regolare $R \in re(\Sigma)$ ha associato un linguaggio $L(R)$:

$$\begin{aligned} \text{caso base: } & \begin{cases} L(R) = \emptyset & \text{se } R = \emptyset \\ L(R) = \{\varepsilon\} & \text{se } R = \varepsilon \\ L(R) = \{a\} & \text{se } R = a \end{cases} \\ \text{caso induttivo: } & \begin{cases} L(R) = L(R_1) \cup L(R_2) & \text{se } R = R_1 \cup R_2 \\ L(R) = L(R_1) \circ L(R_2) & \text{se } R = R_1 \circ R_2 \\ L(R) = (L(R_1))^* & \text{se } R = (R_1)^* \end{cases} \end{aligned}$$

Vediamo qualche esempio di espressione regolare.

Esempi

- $0^*10^* = \{w \mid w \text{ contiene un solo } 1\}$
- $(0 \cup 1) = \{0\} \cup \{1\} = \{0, 1\}$
- $(\Sigma\Sigma)^* = \{w \mid w \text{ ha lunghezza pari}\}$. Prima concatena 2 simboli qualsiasi dell'alfabeto e poi li eleva, quindi avremo sempre $2 * n$ simboli

Teorema

Un linguaggio è regolare **se e solo se** esiste un'espressione regolare che lo descrive, ovvero:

$$\text{REG} \equiv \mathcal{L}(re)$$

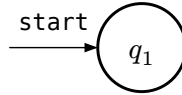
Dimostrazione - Dimostriamo il teorema tramite due lemmi.

1. **Lemma** $\mathcal{L}(re) \subseteq \text{REG}$

Data un'espressione regolare R , costruisco un NFA/DFA N_R t.c. $L(N_R) = L(R)$. Per convertire R in un NFA consideriamo i casi della definizione ricorsiva di R . Prima vediamo i 3 casi base e poi i 3 induttivi.

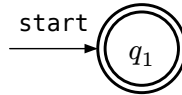
1. $R = \emptyset$. È un NFA che non accetta niente.

- $N_R = \{\{q_1\}, \Sigma, \delta, q_1, \emptyset\}$
- $\delta(q_1, b) = \emptyset, \forall b \in \Sigma$



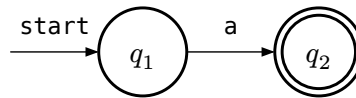
2. $R = \varepsilon$. È un NFA che accetta soltanto la stringa vuota.

- $N_R = \{\{q_1\}, \Sigma, \delta, q_1, \{q_1\}\}$
- $\delta(q_1, b) = \emptyset, \forall b \in \Sigma$



3. $R = a, a \in \Sigma$. È un NFA che accetta soltanto il carattere a .

- $N_R = (\{q_1, q_2\}, \Sigma, \delta, q_1, \{q_2\})$
- $\delta(q_1, a) = q_2$
- $\delta(q, b) = \emptyset, b \neq a$



Passiamo adesso ai 3 casi ricorsivi.

4.5.6. $R = R_1 \cup R_2$; $R = R_1 \circ R_2$; $R = (R_1)^*$. Per costruire i casi induttivi ci basta utilizzare le chiusure dei linguaggi regolari. Ad esempio se dobbiamo costruire $R = R_1 \cup R_2$ ci basta prima costruire i due automi per R_1, R_2 e poi utilizzare la chiusura come visto nelle dimostrazioni precedenti.

Esempio

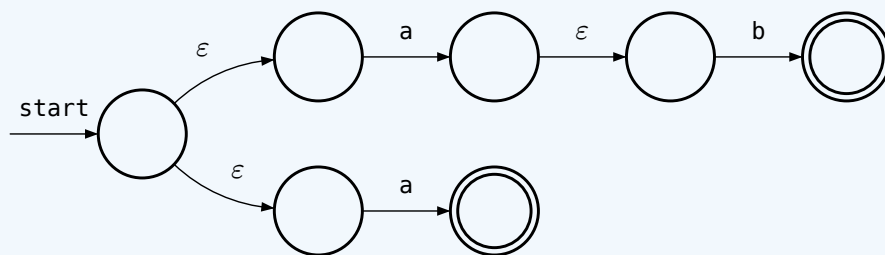
Convertiamo l'espressione regolare $(ab \cup a)^*$ in un NFA: Per prima cosa costruiamo gli automi per riconoscere i caratteri a e b :



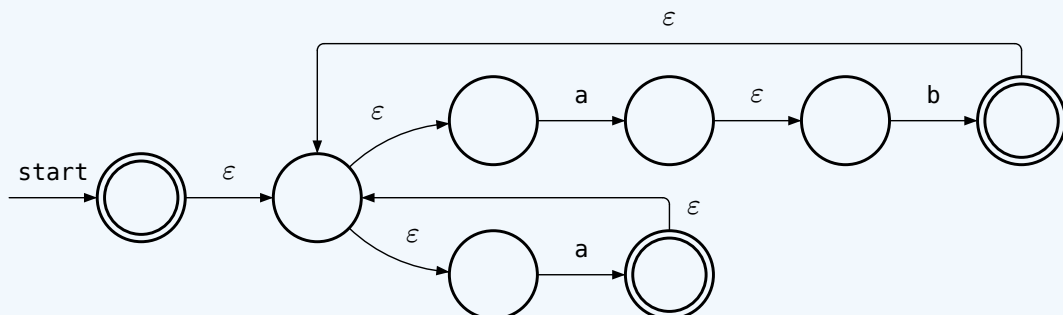
Poi costruiamo quello per riconoscere ab usando appunto la concatenazione:



Adesso usiamo l'unione per costruire l'NFA che riconosce $ab \cup a$:



Possiamo finalmente usare la chiusura per $*$ e costruire l'NFA:



2. **Lemma** $REG \subseteq \mathcal{L}(re)$. Adesso dobbiamo mostrare come passare da un linguaggio regolare ad un'espressione regolare. Per farlo dobbiamo imparare a convertire gli NFA in espressioni regolari.

Per fare questo ci serve il concetto di **NFA Generalizzato (GNFA)**:

- Le etichette degli archi sono espressioni regolari
- Lo stato iniziale ha solo archi uscenti verso ogni altro stato
- C'è un solo stato finale, che ha solo archi entranti da altri stati ed è diverso dallo stato iniziale
- Eccetto gli stati iniziale e finale, presi due stati qualsiasi (anche lo stesso) esiste un arco tra di essi.

Per prima cosa convertiamo l'NFA in un GNFA tramite il seguente algoritmo:

- Se lo stato iniziale ha archi entranti, aggiungere un nuovo stato iniziale con un ϵ -arco al vecchio stato iniziale.
- Se c'è più di uno stato accettante o esiste uno stato accettante con archi uscenti, aggiungere un nuovo stato finale con ϵ -archi provenienti da ogni $q \in F$.

- Se qualche arco ha più di un input, usare come input per quell'arco l'unione tra gli input precedenti.
- Aggiungere archi con input $\emptyset$ tra le coppie di stati non unite da archi.

Possiamo anche fornire una definizione formale di **GNFA**.

Definizione - GNFA

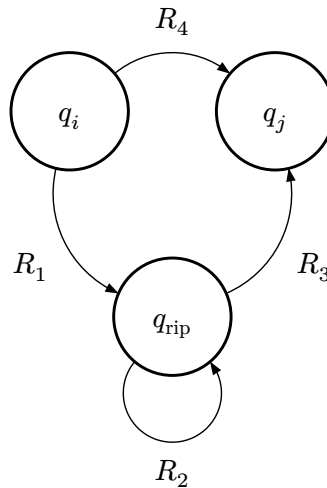
Un GNFA è una tupla $(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$ dove gli elementi sono definiti come negli NFA ad eccezione della funzione di transizione, infatti:

$$\delta : (Q \setminus \{q_{\text{accept}}\}) \times (Q \setminus \{q_{\text{start}}\}) \rightarrow \mathcal{R} = re(\Sigma)$$

Conversione da GNFA a Espressione Regolare - Adesso possiamo convertire il GNFA in espressione regolare. Definiamo la funzione $\text{CONVERT}(G)$ che prende in input un GNFA e restituisce un'espressione regolare:

1. Sia $k = \#\text{stati di } G$
2. Se $k = 2$ allora G ha solo i due stati q_{start} e q_{accept} e un singolo arco con etichetta $R \in \mathcal{R}$, l'output sarà quindi R .
3. Se $k > 2$ scegliamo un $q_{\text{rip}} \in Q$ diverso da q_{start} e q_{accept} . Definiamo $G' = (Q', \Sigma, \delta', q_{\text{start}}, q_{\text{accept}})$ dove:
 - $Q' = Q \setminus \{q_{\text{rip}}\}$
 - $\delta' : Q' \setminus \{q_{\text{acc}}\} \times Q' \setminus \{q_{\text{start}}\} \rightarrow \mathcal{R}$

Quindi prendiamo uno stato q_{rip} a caso e lo rimuoviamo. Come si rimuove? Vediamo un esempio con il seguente automa:



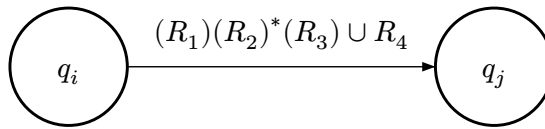
Dobbiamo vedere quali sono tutti i modi che abbiamo per andare da q_i a q_j , ovvero:

- Direttamente fra i due nodi: espressione R_4
- Considerando anche il passaggio per q_{rip} : prima si attraversa R_1 poi R_2 quante volte vogliamo e infine R_3

Scriviamo quindi la nuova espressione che andrà sull'arco da q_i a q_j :

- $\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup R_4$

E possiamo infine ridisegnare il nuovo automa:



4. Si lancia ricorsivamente $\text{CONVERT}(G')$ fino ad arrivare ad un automa con 2 stati.

Adesso però dobbiamo dimostrare che il risultato di $\text{CONVERT}(G)$ è equivalente a G .

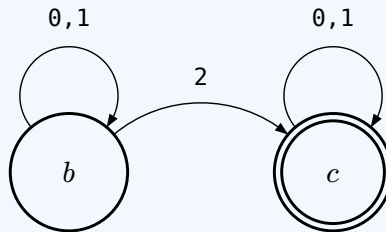
Dimostrazione - Si dimostra per induzione su k numero di stati di G

- **Caso Base:** $k = 2$ è vero perché banalmente ci sono solo due stati e un solo arco che li collega quindi otteniamo lo stesso identico automa applicando la funzione.
- **Passo Induttivo:** Supponendo quindi per $k - 1$ stati valga $G \equiv \text{CONVERT}(G)$, se G accetta $w \in \Sigma^*$ esiste un ramo di computazione t.c. G percorre $q_{\text{start}}, q_1, \dots, q_{\text{acc}}$ e in questo percorso facciamo due distinzioni:
 - Se la sequenza non contiene q_{rip} allora banalmente i linguaggi degli automi sono uguali $L(G) = L(G')$ dato che non abbiamo rimosso nulla dal percorso.
 - Se la sequenza contiene q_{rip} , gli stati adiacenti ad esso che chiamiamo q_1 e q_2 , in G' avranno un nuovo arco che tiene conto di tutti i modi per passare da q_1 a q_2 anche senza q_{rip} e quindi anche in questo caso manteniamo il linguaggio uguale.
- Quindi $G \equiv \text{CONVERT}(G)$ oppure $L(G) = L(G')$

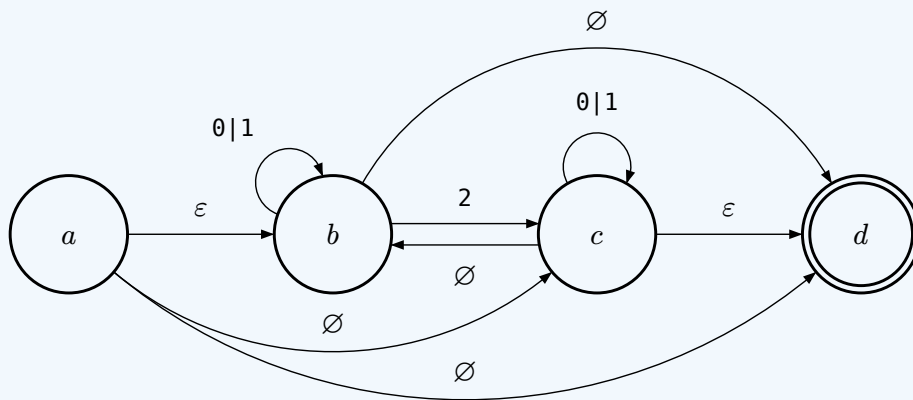
Facciamo un esempio completo per chiarire il procedimento.

Esempio - Conversione da NFA a Esp. Regolare

Dobbiamo trovare l'espressione regolare associata all'automa:



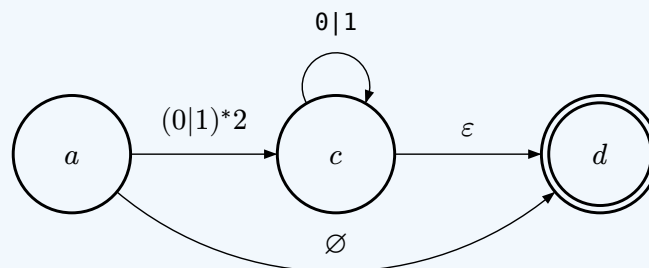
Per prima cosa generalizziamo l'automa:



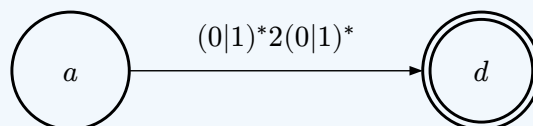
Abbiamo quindi:

- Rimosso archi entranti dallo stato iniziale
- Rimosso archi uscenti dallo stato finale
- Collegato ogni stato con ogni altro stato

Adesso iniziamo la rimozione degli stati, iniziamo rimuovendo b :



Rimuoviamo c :



Otteniamo quindi l'espressione regolare $(0|1)^*2(0|1)^*$

1.7. Pumping Lemma

Il *pumping lemma* ci serve per rispondere alla domanda “tutti i linguaggi sono regolari?”. L’idea dietro il lemma è che tutte le stringhe di un linguaggio regolare hanno una sezione che può essere

ripetuta infinite volte e tutte le stringhe risultanti da questa ripetizione apparterranno ancora al linguaggio.

Teorema - Pumping Lemma

Se L è regolare allora $\exists p$ (*pumping length*) t.c. presa $w \in L$ con $|w| \geq p$ allora w può essere scomposta in $w = xyz$ in modo che:

1. $\forall i \geq 0, xy^iz \in L$
2. $|y| > 0$. *Altrimenti sarebbe banale infatti $y^0 = \epsilon$.*
3. $|xy| \leq p$

Prendiamo come pumping length p il numero degli stati e mostriamo che qualsiasi stringa s di lunghezza maggiore o uguale a p può essere spezzata in xyz . Ovviamente se nessuna stringa è lunga almeno p allora il teorema risulta vero. Se $|s| = n$ allora possiamo dire che gli stati attraversati dall'automa saranno $n + 1$ e dato che $n \geq p$ allora $n + 1 > p$, quindi c'è almeno uno stato che si ripete.

Dividiamo s in xyz in questo modo:

- Individuiamo il primo stato che si ripete e lo chiamiamo q_r
- x è la parte di s che viene prima della **prima apparizione** di q_r .
- y è la parte prima della **prima ripetizione** di q_r .
- z è la parte rimanente della stringa.

Con questa divisione rispettiamo le 3 condizioni:

1. Visto che y ci riporta alla fine di x possiamo ripetere y quante volte vogliamo e quindi se l'automa accetta xyz accetterà anche xy^iz con $i \geq 0$.
2. $|y| > 0$ per costruzione
3. Sappiamo che $|xy| \leq p$ perché y finisce prima della prima ripetizione.

Adesso che abbiamo visto come funziona il lemma "a parole", dimostriamolo in modo formale.

Dimostrazione:

- Sia $M = (Q, \Sigma, \delta, q_1, F)$ t.c. $L(M) = A$ e $p = |Q|$
- Sia $w = w_1 w_2 \dots w_n \in A$ t.c. $n \geq p$
- Siano $r_1, \dots, r_{n+1}$ gli stati attraversati da M durante il processamento di w (quindi la funzione fa i passaggi in modo che $\delta(r_i, w_i) = r_{i+1}$).

La stringa ha lunghezza $n \geq p$ quindi la sequenza di stati è lunga $n + 1 \geq p + 1$ e quindi uno stato appare almeno due volte, chiamiamo le occorrenze di questo stato r_p (la prima) e r_s (la seconda).

Visto che r_s appare tra i primi $p + 1$ (infatti ho p stati ma ne attraverso $\geq p + 1$) stati si ha $s \leq p + 1$.

Adesso siano:

- $x = w_1, \dots, w_{p-1}$
- $y = w_p, \dots, w_{s-1}$
- $z = w_s, \dots, w_n$

Otteniamo che:

1. Visto che x porta M da r_1 a r_p e y porta M da r_p a r_p , M accetterà xy^iz .
2. Sappiamo che $p \neq s$ quindi $|y| > 0$.
3. Sappiamo che $s \leq p + 1$ quindi $|xy| < p$.

Esercizio - Dimostriamo utilizzando il pumping lemma che $L = \{0^n 1^n\}$ non è regolare. Scegliamo $w = 0^p 1^p$ con $p = \text{pumping length}$. Abbiamo $|w| \geq p$. Dato che abbiamo $w = 0^p 1^p$ allora per qualsiasi scomposizione scegliamo $w = xya$ con $|xy| \leq p$ avremo sempre che y è composta da solo “0”:

$$w = \underbrace{00\dots0}_x \underbrace{\dots00}_y 01\dots1$$

Per $i \geq 2$, $\hat{w} = xy^i z$ è $0^q 1^p$ t.c. $q > p$ che non è in L . Quindi il pumping lemma è contraddetto $\Rightarrow L$ non è regolare. Infatti per qualsiasi numero di volte aumenti il numero di 0 non avrò mai lo stesso numero di 1.

Esercizio - Dimostriamo che il linguaggio $L = \{w \in \{0, 1\}^* : \#_0 w = \#_1 w\}$ non è regolare. Prendiamo la stringa $w = 0^p 1^p \in L$ e analizziamo due casi:

1. $\underbrace{0\dots0}_x \underbrace{0\dots0}_y \underbrace{1\dots1}_z$
2. $\underbrace{0\dots0}_x \underbrace{0\dots0}_y \underbrace{001\dots1}_z$

Analizziamoli:

1. Nel primo caso abbiamo che:

- $|y| = k > 0$ con $k \leq p$
- $|x| = p - k$
- $|z| = p$

In questo modo però otteniamo che $|xy^2 z| = (p - k) + 2k + p = 2p + k$ ma allora $\#_0 = (p - k) + 2k = p + k > p$ e quindi $\hat{w} \notin L$.

2. Nel secondo invece:

- $|y| = k > 0$
- $|z| = p + l$. Dove p sono il numero di 1 e l il numero di 0.
- $|x| = p - k - l$. Ovvero tutti gli 0 rimasti.

Adesso però otteniamo che $|xy^2 z| = p - k - l + 2k + p + l = 2p + k$ ma sappiamo che $\#_0 = 2k + p - k - l + l = k + p > p$ e quindi $\hat{w} \notin L$.

Scelta della stringa

È importante scegliere la stringa “giusta” per effettuare la dimostrazione, infatti se ad esempio avessimo scelto $w = (01)^p \in L$ allora non saremmo riusciti a trovare una scomposizione che invalidasse il lemma.

1.8. Linguaggi Acontestuali

I linguaggi acontestuali sono “più potenti” dei linguaggi regolari, quest’ultimi infatti non sono in grado di avere una memoria per, ad esempio, ricordarsi quante occorrenze di un carattere sono state incontrate, questi nuovi linguaggi invece ci permettono di descrivere anche alcune strutture ricorsive.

Introduciamo il concetto di **Grammatica Acontestuale (CFG)**, queste sono formate da regole di sostituzione dette “produzioni” composte da **variabili** (di cui una speciale ovvero quella iniziale) e **terminali**. Vediamo un esempio:

1. $A \rightarrow 0A1$

2. $A \rightarrow B$
3. $B \rightarrow \#$

In questo caso le variabili sono A, B mentre i terminali $0, 1, \#$. Spesso i terminali li indichiamo anche con lettere minuscole.

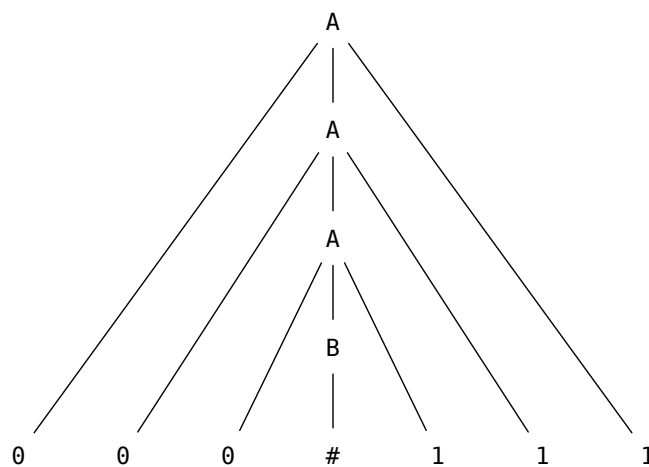
Per generare stringhe con una grammatica acontestuale si usa il seguente procedimento:

1. Si scrive la variabile iniziale
2. Si usano le regole della grammatica per sostituire (a piacimento) le variabili con le espressioni che producono.
3. Si ripete lo step 2 fino a quando non ci sono più variabili.

Esempio - Con la grammatica di prima possiamo generare:

$$A \rightarrow 0A1 \rightarrow 00A11 \rightarrow 000A111 \rightarrow 000B111 \rightarrow 000\#111$$

Per le stringhe generate da una grammatica possiamo disegnare l'**albero di derivazione**:



Definizione - Grammatiche Acontestuali

Una CFG è una tupla (V, Σ, R, S) dove:

- V è un insieme finito di variabili.
- Σ è un insieme finito di terminali ($V \cap \Sigma = \emptyset$)
- R è un insieme finito di regole.
- S è la variabile iniziale.

Notazione

Due o più regole del tipo $A \rightarrow x, A \rightarrow y$, quindi con la stessa variabile che porta a più cose si possono unire per compattezza e scrivere quindi $A \rightarrow x \mid y$.

La terminologia che utilizzeremo invece è la seguente:

- Se $u, v, w \in \Sigma \cup V$ e $(A \rightarrow w) \in R$ diciamo che uAv **produce** uwv e lo scriviamo come $uAv \Rightarrow uwv$.
- Diciamo che u **deriva** w , scritto come $u \Rightarrow^* w$ se:
 - $u = w$, oppure
 - $u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_k \Rightarrow v$

Il **linguaggio di una grammatica** è quindi definito da:

$$\{w \in \Sigma \mid S \Rightarrow^* w\}$$

Esempio

Prendiamo la grammatica $G = (\{S\}, \{a, b\}, R, S)$ dove:

- $R = \{S \rightarrow aSb \mid SS \mid \varepsilon\}$

Allora, ad esempio, $aabb \in L(G)$ infatti:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$$

Esistono diversi modi per costruire una CFG, noi vedremo: **unione, trasformazione da DFA in CFG e ricorsione.**

1.8.1. Unione tra CFG

Supponiamo di avere una CFG $G_i = (V_i, \Sigma_i, R_i, S_i)$, $\forall i \in [k]$ e vogliamo creare una CFG $G = (V, \Sigma, R, S)$ t.c. $L(G) = \bigcup_i L(G_i)$. Allora definiamo:

- $V = \bigsqcup_i V_i \cup \{S\}$. Assumendo quindi (wlog) che $V_i \cap V_j = \emptyset, \forall i \neq j$.
- S nuova variabile iniziale
- $\Sigma = \bigcup_i \Sigma_i$
- $R = \bigcup_i R_i \cup \{S \rightarrow S_1 \mid S_2 \mid \dots \mid S_k\}$

Dimostrazione - Dimostriamo che $\bigcup_i L(G_i) = L(G)$.

- La prima implicazione è $\bigcup_i L(G_i) \subseteq L(G)$. Sia $w \in \bigcup_i L(G_i)$ allora $\exists j \in [k]$ t.c. $w \in L(G_j)$ ovvero $S_j \Rightarrow_{G_j}^* w$, allora per costruzione di G abbiamo che $S \Rightarrow S_j \Rightarrow_{G_j}^* w$ ovvero $w \in L(G)$.
- La seconda implicazione è $L(G) \subseteq \bigcup_i L(G_i)$. Sia $w \in L(G)$ ovvero $S \Rightarrow_G^* w$ allora per costruzione $\exists j \in [k]$ t.c. $S \Rightarrow S_j \Rightarrow_{G_j}^* w$. Siccome V_j è disgiunto da tutti gli altri V_i possiamo dire che:

$$S \Rightarrow \underbrace{S_j \Rightarrow_{G_j}^* w}_{w \in L(G_j) \subseteq \bigcup_i L(G_i)}$$

1.8.2. Trasformare un DFA in un CFG

Dato un DFA $D = (Q, \Sigma, \delta, q_0, F)$ voglio definire $G = (V, \Sigma, R, S)$ t.c. $L(G) = L(D)$, costruiamola nel seguente modo:

- $V = \{V_q \mid q \in Q\}$
- $S = V_{q_0}$
- $\forall q \in F$ aggiungo la regola $V_q \rightarrow \varepsilon$
- $\forall p, q \in Q$ t.c. $\delta(q, a) = p$ aggiungo la regola $V_q \rightarrow aV_p$

Dimostrazione - Dobbiamo dimostrare che $L(D) = L(G)$ quindi vediamo le due implicazioni.

1. $L(D) \subseteq L(G)$. Sia $w = w_1 w_2 \dots w_k \in L(D)$ allora esiste una sequenza di stati

$$\underbrace{q_0, q_1, \dots, q_k, q_{k+1}}_S \text{ con } q_{k+1} \in F$$

tali che $\delta(q_i, w_i) = q_{i+1}, \forall i < k$. Per costruzione possiamo dire che:

- $\forall q_1, q_2 \in S$ t.c. $\delta(q_1, a) = q_2, \exists V_{q_1} \rightarrow aV_{q_2} \in R$.
- $q_{k+1} \in F$, quindi $(V_{q_{k+1}} \rightarrow \varepsilon) \in R$

La stringa viene quindi generata dalla CFG G .

2. $L(G) \subseteq L(D)$. Sia $w \in L(G)$ allora esiste una derivazione $S \Rightarrow^* w$ ma le produzioni in G sono tutte del tipo $(A \rightarrow aB \text{ o } A \rightarrow \varepsilon)$ quindi la derivazione avrà forma

$$V_{q_0} \Rightarrow w_1 V_{q_1} \Rightarrow \dots \Rightarrow w_1 \dots w_k V_{q_k} \Rightarrow w_1 \dots w_k$$

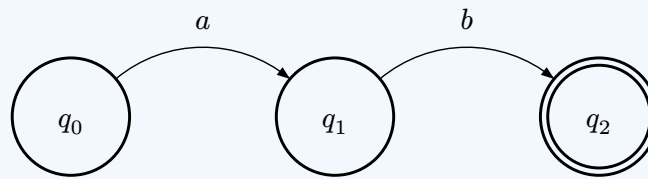
Ogni passo della derivazione è possibile per la regola $(V_{q_i} \rightarrow w_{i+1} V_{q_{i+1}}) \in R$ e per costruzione abbiamo che:

- $\delta(q_i, w_{i+1}) = q_{i+1} \in \delta$
- $(V_{q_k} \rightarrow \varepsilon) \in R \Leftrightarrow q_k \in F$

La stringa viene quindi riconosciuta correttamente dall'automa.

Facciamo un esempio.

Esempio



Costruiamo la grammatica G con:

- $V = \{V_{q_0}, V_{q_1}, V_{q_2}\}$
- $\Sigma = \{a, b\}$
- $R = \{V_{q_0} \rightarrow aV_{q_1}, V_{q_1} \rightarrow bV_{q_2}, V_{q_2} \rightarrow \varepsilon\}$
- $S = V_{q_0}$

Prendiamo la stringa $w = ab \in L(D)$ e vediamo che possiamo infatti generarla con la grammatica:

$$S \Rightarrow aV_{q_1} \Rightarrow abV_{q_2} \Rightarrow ab$$

1.8.3. Forma Normale di Chomsky

Definizione - Forma Normale di Chomsky

Una CFG è in forma normale se ogni regola è del tipo:

- $A \rightarrow BC$
- $A \rightarrow a$
- $S \rightarrow \varepsilon$

Con $A, B, C \in V, a \in \Sigma, B, C \neq S$

Teorema

Ogni CFG ammette una CFG equivalente in forma normale.

Dimostrazione - Facciamo una “dimostrazione” tramite un esempio (?), prendiamo la grammatica con le seguenti regole:

$$R = \{S \rightarrow ASA \mid aB; A \rightarrow B \mid S; B \rightarrow b \mid \varepsilon\}$$

1. Si aggiunge S_0 variabile iniziale insieme alla regola $S_0 \rightarrow S$:

$$R = \begin{cases} S \rightarrow ASA \mid aB \\ A \rightarrow B \mid S \\ B \rightarrow b \mid \varepsilon \\ S_0 \rightarrow S \end{cases}$$

2. Si eliminano le ε -regole del tipo $A \rightarrow \varepsilon$, quindi per ogni occorrenza di A a destra di una regola si aggiunge una nuova regola in cui $A = \varepsilon$:

$$\text{Elimino } B \rightarrow \varepsilon = \begin{cases} S \rightarrow ASA \mid aB \mid a \\ A \rightarrow B \mid S \mid \varepsilon \\ B \rightarrow b \mid \varepsilon \\ S_0 \rightarrow S \end{cases} \quad \text{Elimino } A \rightarrow \varepsilon = \begin{cases} S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S \\ A \rightarrow B \mid S \mid \varepsilon \\ B \rightarrow b \\ S_0 \rightarrow S \end{cases}$$

3. Si eliminano le regole unitarie $A \rightarrow B$ e ogni regola $B \rightarrow x$ con $x \in (V \cup \Sigma)$ viene sostituita da $A \rightarrow x$

$$\text{Elimino } S_0 \rightarrow S \text{ e } S \rightarrow S = \begin{cases} S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid \varepsilon \\ A \rightarrow B \mid S \\ B \rightarrow b \\ S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ S_0 \rightarrow S \end{cases}$$

$$\text{Elimino } A \rightarrow B = \begin{cases} S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ A \rightarrow B \mid S \mid b \\ B \rightarrow b \\ S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS \end{cases}$$

$$\text{Elimino } A \rightarrow S = \begin{cases} S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \\ A \rightarrow S \mid b \mid ASA \mid aB \mid a \mid SA \mid AS \\ B \rightarrow b \\ S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS \end{cases}$$

4. Per ogni regola $A \rightarrow x_1 \dots x_k$ con $k \geq 3$ e $x_i \in (V \cup \Sigma)$ si elimina la regola iniziale e si aggiungono le regole:

$$\begin{cases} A \rightarrow x_1 A_1 \\ A_1 \rightarrow x_2 A_2 \\ \vdots \\ A_{k-2} \rightarrow x_{k-1} x_k \end{cases}$$

$$\text{Otteniamo quindi} = \begin{cases} S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid AA_1 \\ A \rightarrow b \mid ASA \mid aB \mid a \mid SA \mid AS \mid AA_1 \\ B \rightarrow b \\ S_0 \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid AA_1 \\ A_1 \rightarrow SA \end{cases}$$

5. Per ogni regola della forma $A \rightarrow x_1 x_2$, se $x_1 \in \Sigma$ si aggiunge una variabile X_1 ed una regola $X_1 \rightarrow x_1$. Si sostituisce $A \rightarrow x_1 x_2$ con $A \rightarrow X_1 x_2$. Facciamo lo stesso anche se $x_2 \in \Sigma$:

$$\begin{cases} S \rightarrow \cancel{aB} \mid a \mid SA \mid AS \mid AA_1 \mid \color{blue}{XB} \\ A \rightarrow b \mid \cancel{aB} \mid a \mid SA \mid AS \mid AA_1 \mid \color{blue}{XB} \\ B \rightarrow b \\ S_0 \rightarrow \cancel{aB} \mid a \mid SA \mid AS \mid AA_1 \mid \color{blue}{XB} \\ A_1 \rightarrow SA \\ \color{blue}{X} \rightarrow a \end{cases}$$

Abbiamo quindi ottenuto una grammatica equivalente in forma normale di Chomsky:

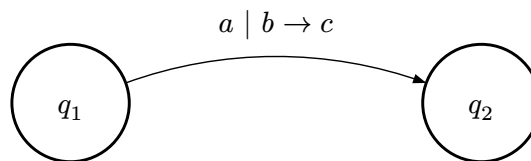
$$\begin{cases} S \rightarrow a \mid SA \mid AS \mid AA_1 \mid XB \\ A \rightarrow b \mid a \mid SA \mid AS \mid AA_1 \mid XB \\ B \rightarrow b \\ S_0 \rightarrow \mid a \mid SA \mid AS \mid AA_1 \mid XB \\ A_1 \rightarrow SA \\ X \rightarrow a \end{cases}$$

1.9. Pushdown Automata - PDA

I PDA sono degli automi in grado di riconoscere i linguaggi delle CFG, questi sono simili agli NFA ma hanno una pila (stack) LIFO. Ad ogni passo di computazione il PDA può svolgere 3 operazioni sulla cima della pila:

- PUSH (inserimento)
- POP (rimozione)
- Sostituzione (POP e poi PUSH in un unico passaggio)

Vediamo un esempio grafico per definire la notazione:



Questo significa che se ci troviamo nello stato q_1 possiamo percorrere l'arco e andare in q_2 soltanto se:

- Leggiamo a
- In cima alla pila abbiamo b
- Inoltre, Per effettuare l'operazione dobbiamo sostituire b con c .

Per PUSH e POP invece scriviamo:

- $\varepsilon \rightarrow x$: PUSH di x sulla cima
- $x \rightarrow \varepsilon$: POP di x dalla cima

Definizione - PDA

Un PDA è una tupla $(Q, \Sigma, \Gamma, \delta, q_0, F)$ dove:

- Q, Σ, q_0, F sono come gli NFA
- Γ è l'alfabeto della pila
- $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q_\varepsilon \times \Gamma_\varepsilon)$

I PDA non deterministici e deterministici sono **equivalenti**!

Vediamo come funzionano le transizioni, dato:

$$(q, c) \in \delta(p, a, b)$$

$$\text{con } p, q \in Q; a \in \Sigma_\varepsilon; b, c \in \Gamma_\varepsilon$$

Abbiamo appunto che p, q indicano gli stati, rispettivamente il precedente e il successivo. a è il carattere letto in input mentre b, c sono i caratteri in cima alla pila, rispettivamente il precedente e il successivo. Abbiamo quindi che:

- Se $a, b, c \neq \varepsilon$ l'automa legge a ed effettua la sostituzione $b \rightarrow c$.
- Se $b = \varepsilon$ l'automa legge a ed effettua il PUSH $\varepsilon \rightarrow c$.
- Se $b \neq \varepsilon$ e $c = \varepsilon$ l'automa legge a ed effettua il POP $b \rightarrow \varepsilon$
- Se $b = c = \varepsilon$ l'automa legge a senza modificare la pila.

Quand'è che un PDA **accetta**? Un PDA accetta una stringa $w = w_1, \dots, w_n$ t.c. $w_i \in \Sigma$ se

$\exists r_0, \dots, r_m \in Q$ e $s_1, \dots, s_m \in \Gamma^*$ t.c.:

- $r_0 = q_o$ (inizia dallo stato iniziale)
- $s_0 = \varepsilon$ (stack vuoto all'inizio)
- $r_m \in F$ (termina in uno stato accettante)
- $\forall i \in [m]$ si ha che:
 - $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$
 - $s_i = at$
 - $s_{i+1} = bt$

Con $a, b \in \Gamma_\varepsilon; t \in \Gamma^*$ stringa della pila.

Accettazione - Da notare che un PDA accetta indipendentemente dal contenuto della pila ma possiamo assumere che la pila debba essere vuota, ad esempio aggiungendo un ε -arco che la svuoti.

Per quanto riguarda le relazioni $\vdash_M$ e $\vdash_M^*$ invece, funzionano come per gli NFA:

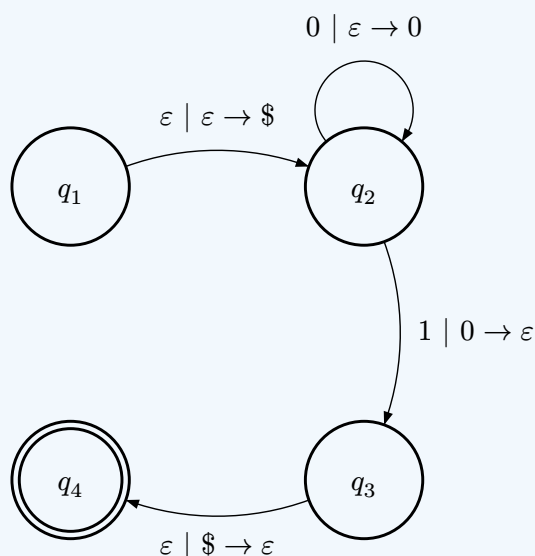
$$L(M) = \{w \in \Sigma^* \mid (q_o, w, \varepsilon) \vdash_M^* (q, \varepsilon, y), q \in F, y \in \Gamma^*\}$$

Come esempio vediamo il precedente linguaggio che abbiamo dimostrato non essere regolare, vediamo quindi come invece un PDA è in grado di riconoscerlo.

Esempi

Costruiamo un PDA per il linguaggio $L = \{0^n 1^n \mid n \geq 0\}$. L'idea è quella di fare PUSH di 0 quando leggiamo 0 e POP di 0 quando leggiamo 1, se la pila è vuota a fine computazione allora accetta. Inseriamo inoltre un "segnalibro" \$ per indicare l'inizio della stringa e riconoscere quindi che è vuota. Abbiamo quindi:

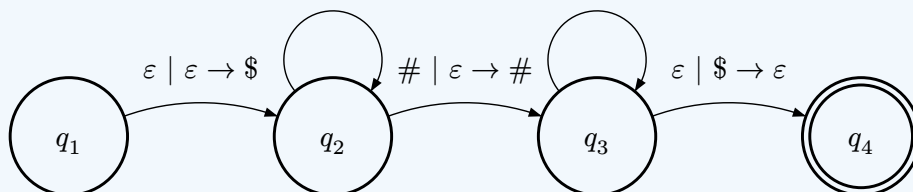
- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, \$\}$
- $F = q_4$
- $q_0 = q_1$



Costruiamo un PDA per il linguaggio $L = \{w\#w^R \mid w \in \{0, 1\}^*\}$. Dobbiamo quindi riconoscere le stringhe composte da una stringa w seguita dal carattere # e poi la stringa w al contrario, ad esempio *mela#alem*. L'idea è quella di scrivere w nella pila fino #, controlliamo i restanti caratteri con quello che abbiamo nella pila:

$$Q = \{q_1, q_2, q_3, q_4\} \quad \Sigma = \{0, 1, \#\} \quad \Gamma = \{0, 1, \$\} \quad F = \{q_4\} \quad q_0 = q_1$$

$$\begin{array}{ll} 0 \mid \varepsilon \rightarrow 0 & 0 \mid 0 \rightarrow \varepsilon \\ 1 \mid \varepsilon \rightarrow 1 & 1 \mid 1 \rightarrow 1 \end{array}$$



Teorema

Un linguaggio è acontestuale **se e solo se** esiste un PDA M che lo riconosce.

Dimostrazione - Per fare questa dimostrazione dividiamo in due **lemmi**.

Lemma - Se un linguaggio è acontestuale allora esiste un PDA M che lo riconosce.

Dimostrazione - Per farlo pensiamo al fatto che se A è un linguaggio acontestuale (CFL) allora esiste una CFG G che lo genera, possiamo convertire questa CFG in un PDA:

- Ad ogni step di derivazione, il non determinismo del PDA P seleziona tutte le regole per una variabile in modo parallelo.
- P inizia scrivendo la variabile iniziale sullo stack e accetta se arriva a una stringa composta solo da terminali e che corrisponde a quella in input.
- Per ogni stringa intermedia il PDA salva solo dalla prima variabile in poi andando a fare un match carattere per carattere dei terminali.

Quello che fa P è quindi:

1. Inserire $\$$ e la variabile iniziale in cima allo stack.
2. Ripete i seguenti passaggi:
 1. Se la cima dello stack è una variabile A la sostituisce in modo non deterministico percorrendo tutte le regole in modo parallelo.
 2. Se la cima è un terminale a legge il prossimo simbolo dell'input e lo confronta con a , se combaciano va avanti altrimenti rifiuta quel ramo di computazione.
 3. Se la cima è $\$$ va allo stato di accettazione.

Iniziamo la **dimostrazione formale**, per farlo però introduciamo una “scorciatoia” ovvero che in un passo di computazione possiamo scrivere sullo stack un'intera stringa invece che un solo simbolo:

$$(r, u) \in \delta(q, a, s)$$

Ovvero P legge a dallo stato q , fa pop di s e scrive la stringa u . Questo è possibile perché equivale ad aggiungere gli stati $q_1, \dots, q_{l-1}$ con $l = |u|$ t.c.:

- $\delta(q, a, s) = (q_1, u_l)$
- $\delta(q_1, \varepsilon, \varepsilon) = (q_2, u_{l-1})$
- $\vdots$
- $\delta(q_l, \varepsilon, \varepsilon) = (r, u_1)$

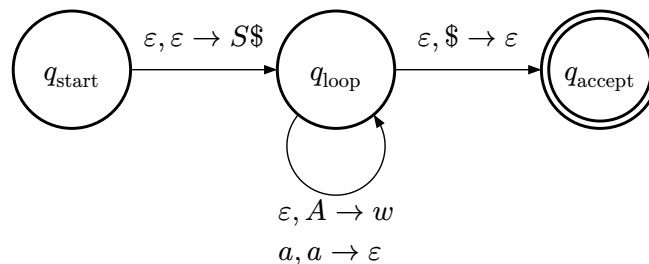
Dato un PDA $P = (Q, \Sigma, \Gamma, \delta, q_{\text{start}}, F)$ con:

- $Q = \{q_{\text{start}}, q_{\text{loop}}, q_{\text{accept}}\} \cup E$. Dove E sono gli stati aggiunti grazie al “trucco” di prima
- δ :
 - Creo il setup iniziale $\delta(q_{\text{start}}, \varepsilon, \varepsilon) = \{(q_{\text{loop}}, S\$)\}$

Poi definiamo 3 regole in base a cosa abbiamo in cima allo stack:

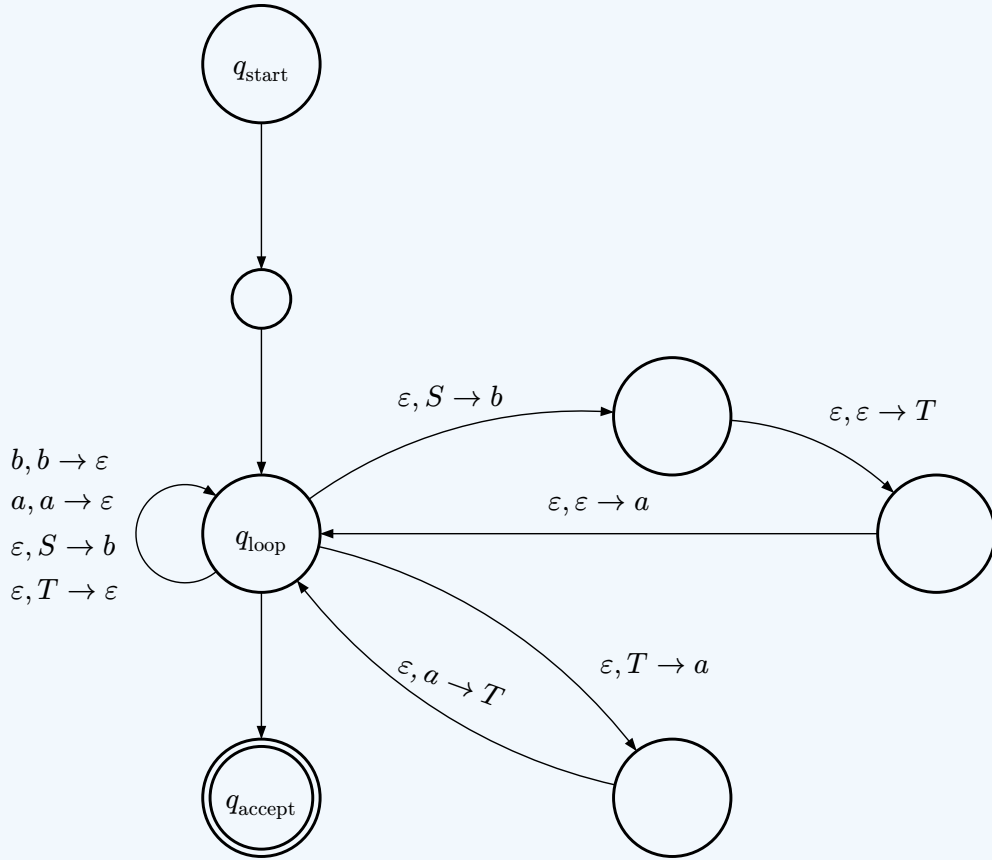
- **Variabile:** $\delta(q_{\text{loop}}, \varepsilon, A) \ni (q_{\text{loop}}, w)$ con $A \rightarrow w \in R$
- **Terminale:** $\delta(q_{\text{loop}}, a, a) = \{(q_{\text{loop}}, \varepsilon)\}$
- **\$:** $\delta(q_{\text{loop}}, \varepsilon, \$) = \{(q_{\text{accept}}, \varepsilon)\}$

Quindi, in generale, abbiamo quest situazione:



Esempio

Prendiamo la grammatica G con le regole $\{(S \rightarrow aTb \mid b), (T \rightarrow Ta \mid \varepsilon)\}$



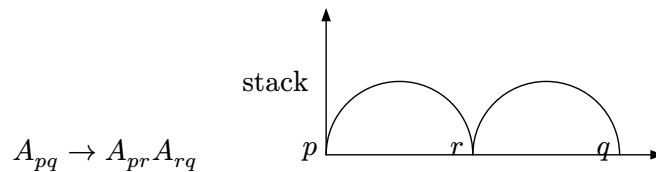
Lemma - Se esiste un PDA P che riconosce L allora L è un linguaggio acontestuale.

Dimostrazione - Costruiamo una CFG che imita M . Assumiamo *wlog* che l'automa P :

- Ha un solo stato accettante q_{acc}
- Svuoti lo stack prima di accettare
- Abbia solo transizioni che fanno PUSH o POP non entrambe.

Per ogni coppia di stati (p, q) la CFG ha una variabile A_{pq} che genera tutte le stringhe che portano M da p a q senza cambiare l'altezza dello stack. Le regole della grammatica si basano sul comportamento della pila nel passare da p a q e ce ne sono di 3 tipi:

- **Stare Fermi:** $\forall p, A_{pp} \rightarrow \varepsilon$. Infatti per andare da p a p senza modificare la pia non dobbiamo leggere nulla.
- Se nell'andare da p a q l'automa passa per un punto r in cui la pila torna vuota allora il passaggio è la somma dei due passaggi. (Ovvero il simbolo pushato all'inizio non è quello che viene pop-ato alla fine).



- Se il simbolo pop-ato alla fine, su input b , è quello pushato all'inizio su input a :

$$A_{pq} \rightarrow aA_{rs}b$$

Formalmente abbiamo che:

- $\forall p, q, r, s \in Q, u \in \Gamma, a, b \in \Sigma_\varepsilon$ se $(r, u) \in \delta(p, a, \varepsilon)$ e $(q, \varepsilon) \in \delta(s, b, u)$ allora mettiamo la regola $A_{pq} \rightarrow aA_{rs}b$ in G
- $\forall p, q, r \in Q$ mettiamo la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ in G .
- $\forall p \in Q$ mettiamo $A_{pp} \rightarrow \varepsilon$ in G .

Claim - A_{pq} genera x **se e solo se** x porta P da p con stack vuoto a q con stack vuoto. Dobbiamo dimostrare entrambi i lati dell'implicazione.

Lato 1 ($\Rightarrow$) - Dimostriamolo per induzione sui k passi di derivazione di x da A_{pq} .

- **Caso base:** La derivazione ha 1 passo allora l'unica regola possibile è $A_{pp} \rightarrow \varepsilon$ che banalmente porta P da p a q con stack vuoto.
- **Passo Induttivo:** Supponendo che sia vero per derivazioni che hanno fino a k passi. Data $A_{pq} \Rightarrow^* x$ con $k + 1$ passi allora il primo passo può essere di 2 tipi:
 - $A_{pq} \Rightarrow aA_{rs}b$ e allora abbiamo $x = ayb$ con y che è stata generata da A_{rs} in k passi e quindi porta P da r a s con stack vuoto. Per costruzione applichiamo $A_{pq} \Rightarrow aA_{rs}b$ se $(r, u) \in \delta(p, a, \varepsilon)$ e $(q, \varepsilon) \in \delta(s, b, u)$
 - $A_{pq} \Rightarrow A_{pr}A_{rq}$ e abbiamo $x = yz$ con y generato da A_{pr} in massimo k passi e z generato da A_{rq} in massimo k passi. Quindi anche qui vale la proprietà di prima.

Lato 2 ($\Leftarrow$) - Dimostriamo per induzione sui k passi di computazione di P da p a q .

- **Caso Base:** Se facciamo 0 step allora la computazione inizia e finisce sullo stesso stato, P non può aver letto nulla per costruzione, allora G ha la regola $A_{pp} \rightarrow \varepsilon$.
- **Passo Induttivo:** Assumendo che sia vero per computazioni lunghe fino a k , ci sono 2 casi:
 - Lo stack è vuoto solo a inizio e fine computazione, allora il simbolo pushato a inizio computazione è lo stesso simbolo pop-ato a fine computazione. Chiamiamo u il simbolo, a l'input letto al primo passo, b l'input dell'ultimo passo, r lo stato dopo il primo passo e s il penultimo. Allora $(r, u) \in \delta(p, a, \varepsilon)$ e $(q, \varepsilon) \in \delta(s, b, u)$ e quindi la regola $A_{pq} \rightarrow aA_{rs}b \in G$.

Possiamo scrivere $x = ayb$ dove y porta P da r a s senza toccare u in $k - 1$ passi, quindi $A_{rs} \Rightarrow^* y$ che comporta $\Rightarrow A_{pq} \Rightarrow^* x$

- Lo stack si svuota durante la computazione. Sia r lo stato in cui lo stack si svuota allora le computazioni che portano P da p a r e da r a q hanno massimo k passi. Quindi scrivendo $x = yz$ abbiamo $A_{pr} \Rightarrow^* y$ e $A_{rs} \Rightarrow^* z$.

Visto che la regola $A_{pq} \rightarrow A_{pr}A_{rq}$ è in G allora $A_{pq} \Rightarrow^* x$.

1.10. Pumping Lemma per i CFL

Dato un CFL L , esiste p t.c. $\forall w \in L$ con $|w| \geq p$ è possibile scomporre w in $w = uvxyz$ in modo che:

1. $\forall i \geq 0, uv^i xy^i z \in L$
2. $|vy| > 0$
3. $|vxy| \leq p$

Dimostrazione - *wlog* Assumiamo che G sia in forma normale di Chomsky, questo ci permette di dire che l'albero di derivazione di w sarà binario, infatti le uniche regole possibili sono $\{(A \rightarrow BC), (A \rightarrow a), (S \rightarrow \varepsilon)\}$.

Claim

Se il cammino più lungo nell'albero ha lunghezza i , la stringa da esso generata ha lunghezza $\leq 2^{i-1}$. Come idea ci basti pensare al caso limite ovvero che ad ogni ramo una variabile si sdoppia in altre due ma ovviamente al livello finale dell'albero tutte le variabili diventano terminali ed è per questo che eleviamo con -1 .

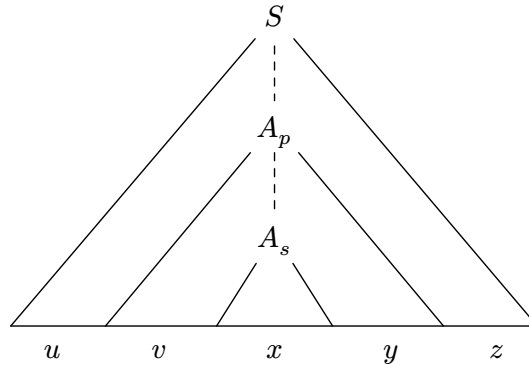
Dimostriamo per induzione:

- **Caso Base:** $i = 1$, l'albero è $S \rightarrow a$ e infatti $|a| = 1 = 2^{1-1} = 2^0 = 1$
- **Passo Induttivo:** Supponendo sia vero fino a $i - 1$, se abbiamo un albero di altezza $i > 1$ la radice S deve avere una regola $S \rightarrow AB$, i due sottoalberi radicati in A e B avranno lunghezza $\leq i - 1$.

Quindi per ipotesi induttiva A e B generano stringhe lunghe al massimo $2^{i-1-1} = 2^{i-2}$. Questo significa che S genera una stringa di lunghezza $\leq 2 \cdot 2^{i-2} = 2^{i-1}$.

Dimostrazione - Sia $m = \#V$ in G poniamo $p = 2^m$. Segue che w t.c. $|w| \geq p$ è generata da un cammino lungo $\geq m + 1$. Il cammino ha quindi almeno $m + 2$ nodi compreso il terminale di cui almeno $m + 1$ variabili. Visto che $|V| = m$ allora almeno una variabile si deve ripetere.

Partendo dalle foglie dell'albero prendiamo la prima variabile che si ripete e chiamiamo A_s la prima occorrenza e A_p la successiva:



E la computazione si divide in $w = uvxyz$

- $S \Rightarrow^* uA_pz$
- $A_p \Rightarrow vA_sy$
- $A_s \Rightarrow x$

Ma sappiamo che $A_p = A_s$ e quindi è come se fosse $A_p \Rightarrow vA_py$ e possiamo quindi sostituire il suo sottoalbero e la derivazione rimarrà uguale:

Definizione - Turing Machine

Una TM è una tupla $M = (Q, \Sigma, \Gamma, \delta, q_{\text{start}}, q_{\text{acc}}, q_{\text{rej}})$ dove:

- Q è l'insieme finito degli stati
- Σ è l'insieme finito di input $\sqcup \notin \Sigma$
- Γ è l'alfabeto del nastro ($\sqcup \in \Gamma, \Sigma \subseteq \Gamma$)
- q_{acc} è lo stato di accettazione
- $q_{\text{rej}} \neq q_{\text{acc}}$ è lo stato di rifiuto
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ definita come, se $\delta(p, a) = (q, b, X)$ allora:
 - a viene letto dal nastro e sostituito da b
 - la macchina passa dallo stato p allo stato q
 - Il nastro viene spostato a sx se $X = L$ e a dx se $X = R$

La transizione avrà quindi come etichetta $a \rightarrow b; X$

Inoltre le transizioni in realtà avvengono da $Q \setminus \{q_{\text{acc}}, q_{\text{rej}}\}$ dato che se la TM va in uno dei due termina subito.

Vediamo quindi adesso il concetto di **configurazione**. La configurazione di una TM si rappresenta in questo modo:

$$uqv$$

Dove:

- La posizione della testina è il primo simbolo di v
- q è lo stato corrente
- u è il restante contenuto del nastro precedente

Quindi ad esempio, una configurazione potrebbe essere:

$$1011q_701111$$

, inoltre la configurazione iniziale è $q_{\text{start}}w$ e quella finale $q \in \{q_{\text{acc}}, q_{\text{rej}}\}$.

Possiamo quindi dire che:

- $uq_i b v \vdash uq_j a c v \Leftrightarrow \delta(q_i, b) = (q_j, c, L)$
- $uq_i b v \vdash uacq_j v \Leftrightarrow \delta(q_i, b) = (q_j, c, R)$

Diciamo che M **accetta** $w \in \Sigma^* \Leftrightarrow c_{\text{start}} = q_{\text{start}}w$ è t.c. $c_{\text{start}} \vdash^* c_{\text{acc}}$ dove c_{acc} è la configurazione di accettazione.

Definizione - Linguaggi Turing Riconoscibili

Un linguaggio L è Turing-Riconoscibile se $\exists$ TM M che lo riconosce, ovvero se $\forall w \in L, M$ accetta w .

Definizione - Decisore

Una TM è detta **decisore** se termina sempre, ovvero non va in loop.

Definizione - Decidibilità

Un linguaggio L è Turing-Decidibile se $\exists M$ decisore t.c. $L = L(M)$

Esistono delle varianti di TM che mantengono lo stesso potere espressivo:

- Nastro infinito a 2 direzioni
- TM che muovono a R, L, S - “stay-put”. Infatti basta che ad ogni transizione fanno prima uno spostamento a sinistra e poi a destra.
- TM con k nastri
- TM non deterministiche

2.1. TM Multinastro

Formalmente abbiamo che $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$ con $k = \#$ nastri, abbiamo quindi

$$\delta(q_i, a_1, \dots, a_k) = (q_j, b_1, b_k, L, R, \dots, L)$$

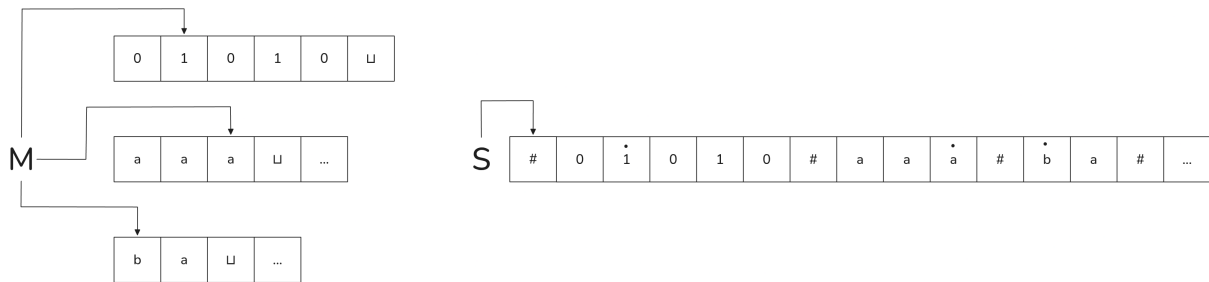
Se la macchina è nello stato q_i e le k testine leggono i simboli $a_1, \dots, a_k$ la macchina va nello stato q_j , scrive i simboli $b_1, \dots, b_k$ e ogni testina si muove come specificato.

Teorema

Ogni TM multinastro ha una TM singolo-nastro equivalente.

Dimostrazione - S simula M mettendo il suo contenuto in un singolo nastro in questo modo:

- Scrive sequenzialmente i contenuti dei k nastri con $\#$ come delimitatore
- Per tenere traccia di dove si trovano le testine, introduce un simbolo $\circ$ sopra al carattere corrispondente, le chiamiamo “testine virtuali”.



Quindi S su input $w = w_1, \dots, w_n$ si comporta così:

1. Pone il nastro nella configurazione “iniziale” $\# w_1^\circ w_2 \dots w_n \# \sqcup^\circ \# \dots \#$
2. S scansiona il nastro dal primo all’ultimo $\#$, per capire quali siano i simboli puntati dalle testine virtuali. Poi fa una seconda scansione per aggiornare il nastro secondo δ' di M .
3. Se in qualsiasi momento una testina finisce su un $\#$ significa che ha raggiunto un $\sqcup^\circ$ e quindi vi scrive $\sqcup'$ e sposta di una posizione a destra l’intero contenuto del nastro.

Corollario

Un linguaggio è Turing-Riconoscibile $\Leftrightarrow$ una TM multinastro lo riconosce.

Dimostrazione - Dato che la singolo-nastro è una caso speciale di multinastro e che ogni multinastro ha una singolo-nastro equivalente la dimostrazione è completa.

2.2. TM non Deterministica

Formalmente abbiamo che:

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

Il concetto di non determinismo è uguale a quello che troviamo negli NFA. Semplicemente in questo caso quando la TM deve fare una mossa potrebbe avere più azioni possibili da svolgere e come per gli NFA, creiamo più percorsi indipendenti.

Simuliamo una NTM N con una TM D , per farlo consideriamo l'albero di computazione di N che esploriamo con una BFS. Non effettuiamo la ricerca con una DFS perché dato che vogliamo simulare il parallelismo se eseguiamo una DFS questa potrebbe continuare all'infinito su un ramo.

L'albero ha questa struttura:

- La radice è ε
- Ogni altro nodo ha un indirizzo $\in \Gamma_b = \{1, \dots, b\}$ con b il massimo numero di scelte tra tutti gli stati di N , nello specifico l'indirizzo è formato da xy con x indirizzo del padre e $y \in \Gamma_b$ "numero" del figlio, quindi ad esempio 231 indica il secondo figlio della radice $\rightarrow$ il suo terzo figlio $\rightarrow$ il suo primo figlio.

La TM D che simula N è una multi-nastro con 3 nastri:

1. Il nastro 1 contiene l'input w e non viene mai alterato. Ci serve come riferimento per quando cambiamo ramificazione.
2. Il nastro 2 simula un ramo della computazione di N , quando ha più scelte effettua quella presente in 3.
3. Il nastro 3 contiene l'indirizzo del nodo fino a cui simulare, quindi conterrà ad esempio "1", "2", "3" poi passa ai figli del primo nodo quindi "11", "12", "13" e poi a quelli del primo nodo ancora "111", "112"...

La TM D si comporta quindi in questo modo:

1. Inizialmente il nastro 1 contiene w e gli altri 2 sono vuoti.
2. Copia il nastro 1 sul nastro 2
3. Usa il nastro 2 per simulare N con input w . Ad ogni step N consulta il prossimo simbolo sul nastro 3
 - Se il nastro 3 è vuoto va allo step 4
 - Se la simulazione rifiuta va allo step 4
 - Se la simulazione accetta, accetta
4. Sostituisce la stringa sul nastro 3 con la successiva in ordine lessicografico. Va al passo 2.

Esempio

Una TM non deterministica può essere una TM che su input binario:

- input 0:
 - Si muove a dx
 - Si muove a sx
- input 1: accetta

Quindi quando legge uno 0 crea due rami, se simuliamo questa TM non deterministica con una deterministica avremo che eseguirà entrambe le scelte prima o poi.

Tesi di Church-Turing

Qualunque problema calcolabile da un algoritmo può essere risolto da una macchina di Turing.

2.3. Linguaggi Decidibili

Sono quei linguaggi per i quali esiste una TM che per ogni stringa termina sempre in un tempo finito e stabilisce se questa appartiene o no al linguaggio.

Problema dell'accettazione per i DFA - Definiamo

$$A_{\text{DFA}} = \{ \langle B, w \rangle \mid B \text{ è un DFA che accetta } w \}$$

E presentiamo una TM che accetta A_{DFA} :

M = "Su input $\langle B, w \rangle$, con B DFA e w stringa:

- Simula B su input w
- Se la simulazione finisce con uno stato di accettazione, accetta; altrimenti rifiuta."

Problema dell'accettazione per gli NFA - Definiamo

$$A_{\text{NFA}} = \{ \langle B, w \rangle \mid B \text{ è un NFA che accetta } w \}$$

E presentiamo una TM che accetta A_{NFA} :

N = "Su input $\langle B, w \rangle$, con B NFA e w stringa:

- Converti B in un DFA C
- Esegui la TM 'M' (per A_{DFA}) su C

-Se M accetta, accetta; altrimenti rifiuta."

Determinare se una regex genera una stringa - Definiamo

$$A_{\text{REX}} = \{ \langle R, w \rangle \mid R \text{ regex che genera } w \}$$

E costruiamo una TM che accetta A_{REX} :

P = "Su input $\langle R, w \rangle$ con R regex e w stringa:

- Converti R in NFA A
- Esegui la TM N (per A_{NFA} su A)
- Se N accetta, accetta; altrimenti rifiuta"

Emptiness-testing per i DFA - Definiamo

$$E_{\text{DFA}} = \{ \langle A \rangle \mid A \text{ DFA e } L(A) = \emptyset \}$$

E costruiamo una TM che accetta E_{DFA} :

T = "Su input $\langle A \rangle$ con A DFA:

- Marca lo stato iniziale
- Finchè non vengono marcati nuovi stati, marca ogni stato che ha una transizione entrante da uno stato già marcato.
- Se almeno uno stato accettante è stato marcato, rifiuta; altrimenti accetta."

Determinare se due DFA accettano lo stesso linguaggio - Definiamo

$$EQ_{\text{DFA}} = \{ \langle A, B \rangle \mid A \text{ e } B \text{ DFA e } L(A) = L(B) \}$$

Utilizziamo la differenza simmetrica:

$$L(C) = L(A) \triangle L(B)$$

, se è vuota abbiamo che $L(A) = L(B)$. Si ha

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

Dobbiamo verificare se $\langle C \rangle \in E_{\text{DFA}}$, costruiamo

F = "Dato un input:

- Costruisci il DFA C come descritto
- Simula la TM T (per E_{DFA})
- Se T accetta, accetta; altrimenti rifiuta“

Determinare se una CFG genera una stringa - Definiamo

$$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ CFG genera } w \}$$

Costruiamo la TM

$M =$ “Su input $\langle G, w \rangle$ con G CFG e w stringa:

- Converti G in CNF (Chomsky)
- Enumera le derivazioni con $2n - 1$ passi dove $n = |w|$, se $n = 0$ enumera solo quelle con 1 passo. Da questa possiamo ricavare il successivo *claim*
- Se una delle derivazioni genera w , accetta; altrimenti rifiuta.“

Claim - Sia $G = (V, \Sigma, R, S)$ una CFG in CNF. Data $w \in G$ con $|w| = n$, la sua derivazione è di $2n - 1$ passi.

Dimostrazione - Dimostriamolo per induzione su n .

- Caso Base: $n = 1$ allora $|w| = 1$ significa che $w \in \Sigma$. Quindi w è derivata con la regola $S \rightarrow w$ in $1 = 2 \cdot 1 - 1$ passi.
- Passo Induttivo: Assumiamo sia vero per stringhe di lunghezza $\leq n - 1$. Sia $w \in G$ t.c. $|w| = n$. Visto che G è in CNF, $S \xRightarrow{*} w$ implica che $S \rightarrow AB \xRightarrow{*} w$. Quindi possiamo scrivere $w = xy$ con $A \xRightarrow{*} x$ e $B \xRightarrow{*} y$. Visto che non ci sono ε -regole in CNF abbiamo che $x, y \neq \varepsilon$. Quindi siano $|x| = k$ e $|y| = n - k$, per ipotesi induttiva x, y sono derivabili in $2k - 1$ e $2(n - k) - 1 = 2n - 2k - 1$ produzioni. Quindi $AB \xRightarrow{*} w$ ha $2k - 1 + 2n - 2k - 1 = 2n - 2$ produzioni, aggiungendo $S \rightarrow AB$ otteniamo che $S \xRightarrow{*} w$ in $2n - 1$ produzioni.

Emptiness-testing per CFG - Definiamo

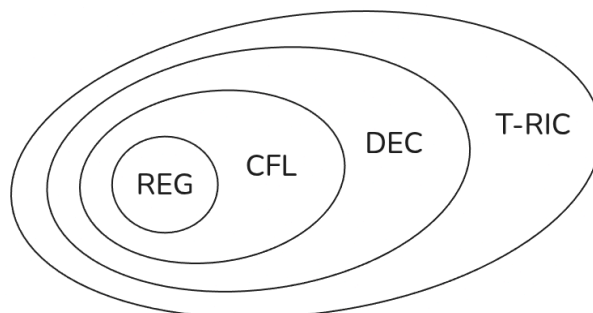
$$E_{CFG} = \{ \langle G \rangle \mid G \text{ CFG e } L(G) = \emptyset \}$$

Costruiamo la TM

$M =$ “Su input G , dove G è una CFG:

- Marca tutti i terminali in Σ
- Finchè non vengono marcate nuove variabili:
 - Marca ogni $A \in V$ per cui $\exists A \rightarrow U_1 U_2 \dots U_l \in R$ dove ogni U_i è già stato marcato
- Se S è marcata, rifiuta; altrimenti accetta“.

Otteniamo quindi questa situazione per i linguaggi:



Esistono però linguaggi non TURING-DEC / RIC, introduciamo quindi il problema A_{TM} che determina se una TM accetta un determinato input.

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ TM e } w \in L(M) \}$$

Definizione - Turing Machine Universale

Una TM universale è una TM in grado di simulare ogni altra TM. Si dice infatti che un modello di calcolo è Turing-complete se è equivalente ad una TM universale.

Una qualsiasi TM M può essere completamente descritta da un insieme finito di transizioni, e quindi può essere codificata come una stringa finita di simboli. Una TM universale prende in input la descrizione codificata della TM da simulare $\langle M \rangle$ e l'input w su cui dovrebbe lavorare e simula M .

Notiamo che A_{TM} è *T-REC*, infatti abbiamo la TM U che lo riconosce:

$U =$ “Su input $\langle M, w \rangle$ con M TM e w stringa:

- Simula M su w
- Se M raggiunge uno stato accettante, accetta; altrimenti rifiuta“

Con questa costruzione se M va in loop ci andrà anche U .

Teorema

A_{TM} non è decidibile.

Ma per dimostrarlo abbiamo prima bisogno di un altro teorema.

Teorema

Esistono linguaggi non Turing-riconoscibili.

Dimostrazione - Ogni TM M può essere descritta con un encoding $\langle M \rangle$ rappresentato da una stringa appartenente ad Σ^* di un alfabeto Σ . L'insieme delle stringhe di un alfabeto è numerabile, siccome tra TM e stringhe c'è una relazione biunivoca allora anche l'insieme $\mathcal{M}$ delle TM è numerabile.

Definiamo adesso $\mathcal{B}$ come l'insieme delle stringhe binarie infinite, questo è non numerabile ed è dimostrabile tramite *Diagonale di Cantor*. Consideriamo anche $\mathcal{L} = \mathcal{P}(\Sigma^*)$ l'insieme di tutti i linguaggi definiti su Σ , anche questo non è numerabile e lo mostriamo fornendo una corrispondenza biunivoca tra l'insieme $\mathcal{B}$ e l'insieme $\mathcal{L}$.

Definiamo il concetto **Sequenza Caratteristica** di A , ovvero una sequenza binaria definita come segue:

$$\mathcal{X}_A = b_1 b_2 \dots \text{ t.c. } b_i = \begin{cases} 1 & s_i \in A \\ 0 & s_i \notin A \end{cases}$$

Esempio

Prendiamo il linguaggio $A =$ “Stringhe binarie che iniziano con 0”:

- $\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, \dots\}$
- $A = \{, 0, , 00, 01, , , \dots\}$
- $\mathcal{X}_A = 0101100\dots$

Notazione

La notazione utilizzata per le funzione indica:

$$\text{nome funzione} : \text{dominio} \rightarrow \text{codominio} : \text{input} \mapsto \text{output}$$

Notiamo che $f : \mathcal{L} \rightarrow \mathcal{B} : A \mapsto \mathcal{X}_A$ è biunivoca, perciò visto che $\mathcal{B}$ non è numerabile non lo è neanche $\mathcal{L}$.

Quindi la funzione $h : \mathcal{M} \rightarrow \mathcal{L} : M \mapsto L(M)$ non è biiettiva (dato che non può essere suriettiva infatti le TM sono numerabili ma i linguaggi no), quindi $\exists L \in \mathcal{L}$ t.c. $\nexists M$ per cui $L(M) = L$.

Adesso possiamo tornare a dimostrare che A_{TM} non è decidibile.

Dimostrazione - Supponiamo per assurdo che $A_{TM} \in DEC$ e sia H un decisore per A_{TM} :

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{se } M \text{ accetta } w \\ \text{reject} & \text{se } M \text{ non accetta } w \end{cases}$$

Adesso costruiamo una TM

$D =$ "Su input $\langle M \rangle$ con M TM:

1. Esegui H con input $\langle M, \langle M \rangle \rangle$
2. Dai in output l'opposto dell'output di H"

$$D(\langle M \rangle) = \begin{cases} \text{accetta} & \text{se } M \text{ non accetta } \langle M \rangle \\ \text{rifiuta} & \text{se } M \text{ accetta } \langle M \rangle \end{cases}$$

Ma adesso cosa succede se eseguiamo D con la sua stessa descrizione $\langle D \rangle$ in input?

$$D(\langle D \rangle) = \begin{cases} \text{accetta} & \text{se } D \text{ non accetta } \langle D \rangle \\ \text{rifiuta} & \text{se } D \text{ accetta } \langle D \rangle \end{cases}$$

Ma abbiamo ottenuto una contraddizione, se D accetta $\langle D \rangle$ allora D deve rifiutare $\langle D \rangle$ e viceversa.

Definizione - Coturing-Riconoscibile

Un linguaggio si dice *Co-Turing-Riconoscibile* se è il complemento di un linguaggio *Turing-Riconoscibile*

Teorema

Un linguaggio L è decidibile $\Leftrightarrow L \in REC$ e $L \in CO-REC$ (ovvero $\bar{L} \in REC$)

Dimostrazione

- $\Rightarrow$ se L è decidibile, $\bar{L}$ è decidibile quindi L e $\bar{L}$ sono *Turing-Riconoscibili*. Questo perché siccome L è decidibile allora esiste una TM decisore per L , se a questa TM decisore invertiamo gli input sarà un decisore per $\bar{L}$ e quindi anche $\bar{L}$ è decidibile e quindi riconoscibile.
- $\Leftarrow$ sia M_1 un riconoscitore per A e M_2 un riconoscitore per $\bar{A}$

Costruiamo $M =$ "Su input w :

1. Esegui M_1 e M_2 in parallelo su w
2. Se M_1 accetta, accetta; se M_2 accetta, rifiuta"

Ogni stringa w è in A oppure in $\bar{A}$, quindi uno tra M_1 e M_2 deve accettare. Visto che M si ferma se M_1 o M_2 accettano, termina sempre e quindi è un decisore. Inoltre, accetta le stringhe in A e rifiuta quelle in $\bar{A}$ quindi è un decisore per A .

Corollario 1

Se L non è decidibile allora almeno uno tra L e $\bar{L}$ non è *Turing-Riconoscibile*. Per vederlo basta negare tutto il teorema e ottenere:

$$L \notin \text{DEC} \Leftrightarrow L \notin \text{REC} \vee \bar{L} \notin \text{REC}$$

Corollario 2

Seguendo quindi il corollario precedente, siccome A_{TM} non è decidibile e A_{TM} è riconoscibile allora $\bar{A}_{\text{TM}}$ non è riconoscibile.

2.4. Riducibilità

Vediamo una tecnica per dimostrare che alcuni problemi non sono decidibili o riconoscibili. Una **riduzione** è un modo di convertire un problema in un altro problema in modo che una soluzione per il secondo problema possa essere usata per il primo. *(Ci serve aumentare il modello delle TM aggiungendo un nastro di output WLOG)*

Diremo che una TM calcola una funzione su un dato input se inizia con w sul nastro di input e termina con $f(w)$ sul nastro di output.

Definizione - Calcolabilità

Una funzione $f : \Sigma^* \rightarrow \Sigma^*$ è calcolabile se $\exists \text{TM } M$ che calcola $f \forall w \in \Sigma^*$

Definizione - Mapping Reductions

A è riducibile tramite funzione a B , lo indichiamo con $A \leq_m B$, se $\exists f : \Sigma^* \rightarrow \Sigma^*$ calcolabile t.c. $\forall w \in \Sigma^*$ si ha che $w \in A \Leftrightarrow f(w) \in B$.

Possiamo leggere il simbolo come “non è più difficile di”, quindi se abbiamo una soluzione per B allora vale anche per A .

Teorema

Se $A \leq_m B$ e B decidibile allora A decidibile.

Dimostrazione Per ipotesi abbiamo che:

- $A \leq_m B$ quindi esiste una funzione calcolabile che fa la riduzione, siccome la funzione è calcolabile esiste una TM F che la calcola.
- B è decidibile quindi esiste un decisore M che lo riconosce.

Costruiamo una terza TM M' per A che su input w :

1. Usa F con input w per calcolare $f(w)$
2. Lancia M con input $f(w)$ e accetta se quest'ultima accetta; altrimenti rifiuta.

Notiamo che:

- Se $w \in A$ allora F calcola correttamente $f(w)$ e poi sia M che M' accetteranno la stringa dato che $f(w) \in B$.
- Al contrario se $w \notin A$ allora F calcolerà correttamente $f(w)$ ma $\notin B$ e quindi entrambe le TM rifiuteranno.
- I due casi sono corretti e inoltre M' utilizza due decisor, questo significa che anche lui impiega un tempo finito e termina sempre, è anch'esso un decisore per A .

Abbiamo costruito un decisore per A quindi A è decidibile.

Corollario

Se $A \leq_m B$ e A è indecidibile allora B è indecidibile

Dimostrazione - Se B fosse decidibile allora A sarebbe decidibile per il teorema.

2.5. Halting Problem

Serve a determinare se una TM termina.

$$\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ TM e } M \text{ termina su input } w \}$$

Teorema

HALT_{TM} è indecidibile.

Dimostrazione - Assumiamo che esista una TM R che decide HALT_{TM} . Usiamo R per costruire S decisore per A_{TM} che sappiamo essere indecidibile.

Supponiamo quindi che $\exists$ TM R che decide HALT_{TM} . Costruiamo S decisore per A_{TM} :

S = Su input $\langle M, w \rangle$ con M encoding di una TM e w stringa:

1. Esegui R su $\langle M, w \rangle$
2. Se R rifiuta, rifiuta
3. Se R accetta, simula M su w fino alla sua terminazione
 - Se M accetta, accetta; altrimenti rifiuta.

Tramite il passaggio 2 evitiamo che S vada in loop, abbiamo quindi costruito un decisore per A_{TM} ma è impossibile, contraddizione.

Quello che abbiamo fatto in questo caso è stato fare una riduzione $A_{\text{TM}} \leq_m \text{HALT}_{\text{TM}}$

In generale la regola è utilizzare nuovi problemi per risolvere problemi che già sappiamo essere non risolvibili, in questo modo creiamo il paradosso e dimostriamo la non decidibilità con la riduzione.

2.6. Altri linguaggi non decidibili

Teorema

$$E_{\text{TM}} = \{ \langle M \rangle \mid M \in \text{TM} \wedge L(M) = \emptyset \} \notin \text{DEC}$$

Dimostrazione - Supponiamo per assurdo che sia decidibile, esiste quindi un decisore:

$$R(\langle M \rangle) = \begin{cases} \text{accetta} & \text{se } L(M) = \emptyset \\ \text{rifiuta} & \text{se } L(M) \neq \emptyset \end{cases}$$

Utilizziamo questo decisore per costruire un decisore S per A_{TM} , questo però in input vorrà $\langle M, w \rangle$ con M TM e w stringa ma R riceve soltanto $\langle M \rangle$ come input.

La TM S dovrà creare sul suo nastro la codifica di una TM M' che:

- Su input x :
 - Ignora x
 - Simula la TM originale M su w
 - Se M accetta w , accetta; altrimenti rifiuta.

Quindi il linguaggio riconosciuto da questa TM M' dipende da cosa fa M con input w :

- Se M accetta, allora M' indipendentemente da x accetterà sempre, il suo linguaggio è quindi infinito.
- Se M non accetta, allora M' indipendentemente da x rifiuterà sempre e quindi il suo linguaggio è vuoto.

La TM S prende questa codifica $\langle M' \rangle$ e la passa ad R :

- Se R accetta significa che il linguaggio di M' è vuoto ma questo significa che M non ha accettato w e quindi S deve rispondere rifiuta per essere decisore di A_{TM} .
- Se R non accetta significa che il linguaggio di M' non è vuoto ma questo significa che M ha accettato w e quindi S deve rispondere accetta per essere decisore di A_{TM} .

Notiamo quindi che S riconosce A_{TM} e inoltre termina sempre dato che usa R che è un decisore e costruisce M' in un tempo finito. Questo significa che S è un decisore per A_{TM} che sappiamo essere indecidibile. *assurdo*.

Teorema

$$REG_{TM} = \{ \langle M \rangle \mid M \text{ TM} \wedge L(M) \in REG \} \notin DEC$$

Dimostrazione - Supponiamo che sia decidibile ed esiste quindi un decisore R che lo decide: $R(\langle M \rangle) = \begin{cases} \text{accetta} & \text{se } L(M) \in REG \\ \text{reject} & \text{se } L(M) \notin REG \end{cases}$

Usiamo questo decisore R per costruire un decisore S per A_{TM} , su input $\langle M, w \rangle$ con M TM e w stringa:

- Crea sul suo nastro l'encoding di una TM M' con la seguente logica:
 - Su input x
 - Se $x \in \{0^n 1^n\}$ accetta
 - Altrimenti simula M originale su w
 - Se M accetta, accetta; altrimenti rifiuta

Di che tipo è il linguaggio di $L(M')$?

- Se M accetta w significa che nel linguaggio finiranno tutte le stringhe, infatti se una stringa w rispetta il punto 1 viene accettata ma se non lo rispetta viene accettata successivamente nel punto dopo. Il linguaggio che otteniamo è quindi Σ^* che è regolare.
- Se M non accetta w il linguaggio sarà $\{0^n 1^n\}$ che sappiamo essere non regolare.

A questo punto S prende la codifica $\langle M' \rangle$ e la fornisce in input a R :

- Se R accetta, significa che $L(M')$ è regolare ma questo accade soltanto se M accetta w e se questo accade S deve restituire accetta.
- Se R rifiuta, significa che $L(M')$ non è regolare ma questo accade soltanto se M non accetta w e se questo accade S deve restituire rifiuta.

A questo punto abbiamo costruito un riconoscitore per A_{TM} ma notiamo che questo termina sempre dato che usa R decisore e costruisce una codifica in tempo finito. Siccome abbiamo costruito quindi un decisore per A_{TM} abbiamo raggiunto un *assurdo*.

2.7. I Teoremi di Gödel

Introduciamo il concetto di **sistema di prova**

Definizione - Sistema di Prova

Un sistema di prova Π è tale che:

- Per ogni affermazione vera o falsa esiste una sua rappresentazione come stringa $\langle x \rangle$ finita.
- Per ogni dimostrazione esiste una rappresentazione come stringa $\langle w \rangle$ finita
- Esiste TM V decisore tale che $\forall (\langle x, w \rangle) \in \Sigma^*$ si ha $\langle x, w \rangle \in L(V) \Leftrightarrow w$ è una dimostrazione per x in Π

Un'affermazione $\langle x \rangle$ è detta **dimostrabile** se $\exists \langle w \rangle \in \Sigma^*$ tale che $\langle w, x \rangle \in L(V)$.

Si dice invece, **indipendente** se né x né $\neg x$ sono dimostrabili in Π .

Π deve essere **computabile** ovvero $\forall A$ assioma in Π , $A \in DEC$

Infine un sistema di prova Π è detto:

- **Consistente**: Non esiste nessuna $\langle x \rangle$ per cui il sistema sia in grado di dimostrare sia x che $\neg x$.
(Non si contraddice mai)
- **Valido**: Se ogni affermazione dimostrabile è vera. (Non dice bugie)
- **Completo**: Se $\forall \langle x \rangle$ almeno uno tra x e $\neg x$ è dimostrabile.
- **Incompleto**: Se esiste un'affermazione indipendente.

Osservazioni

- Valido $\Rightarrow$ consistente
- Π consistente e completo $\Rightarrow$ Per ogni x solo uno tra x e $\neg x$ è dimostrabile
- Π valido e completo $\Rightarrow$ Per ogni x solo uno tra x e $\neg x$ è dimostrabile e quindi vero

Teorema

Sia Π un sistema di dimostrazione abbastanza potente da comprendere l'aritmetica allora Π non può essere sia valido che completo.

Dimostrazione - Sia $L = \{\langle x \rangle \mid x \text{ è dimostrabile in } \Pi\}$ definiamo la TM R_Π che prova a decidere L :

R_Π = Data la stringa $\langle x \rangle$ in input:

- $\forall k = 1, 2, 3, \dots$:
 - $\forall w \in \Sigma^*$ con $|w| = k$:
 - Se $V(\langle x, w \rangle) = ACC$ allora R_Π accetta
 - Se $V(\langle \neg x, w \rangle) = ACC$ allora R_Π rifiuta

Ora supponiamo per assurdo Π valido e completo. Allora una tra x e $\neg x$ deve essere vera, allora abbiamo che:

- x è vera $\Leftrightarrow \exists \text{ dim } w \text{ t.c. } V_\Pi(\langle x, w \rangle) = ACC$ quindi $\langle x \rangle \in L(R_\Pi)$
- $\neg x$ è vera $\Leftrightarrow \exists \text{ dim } w \text{ t.c. } V_\Pi(\langle \neg x, w \rangle) = ACC$

Quindi R_Π è un decisore e $L(R_\Pi) = \{\langle x \rangle \mid x \text{ vera}\}$, grazie alle ipotesi.

Data una TM M e un input $y \in \Sigma^*$ consideriamo la seguente affermazione:

$$\varphi_{M,y} = M(y) \text{ termina}$$

Definiamo la TM $D(< M, y >)$ che passa l'equazione $\varphi_{M,y}$ a R_Π e:

- Se M termina allora R_Π trova la dimostrazione e accetta, anche D accetta.
- Se M non termina allora R_Π trova la dimostrazione per $\neg\varphi_{M,y}$ e rifiuta, D rifiuta.

A questo punto abbiamo costruito la TM D che riesce a risolvere il problema dell'arresto HALT_{TM} , riesce a dire se la macchina M in input si ferma su l'input y . Siccome sappiamo che HALT_{TM} non è decidibile abbiamo raggiunto un *assurdo*.

Lemma

Sia Π un sistema di dimostrazione abbastanza potente da comprendere l'aritmetica. Sia M una TM e $x \in \Sigma^*$ un input.

Se $M(x)$ ha una traccia di esecuzione il cui comportamento è noto allora esiste una dimostrazione in Π che la descrive.

2.7.1. Primo Teorema di Incompletezza

Sia Π un sistema di dimostrazione abbastanza potente da comprendere l'aritmetica. Se Π è consistente allora Π non è completo.

Dimostrazione - L'idea è che se Π è abbastanza potente da comprendere l'aritmetica allora può descrivere il funzionamento di una TM.

Consideriamo l'affermazione $\varphi_M = M(< M >)$ termina

Definiamo la TM R_Π che data la stringa $< x >$ in input:

- $\forall k = 1, 2, 3, \dots$
 - $\forall w \in \Sigma^*$ t.c. $|w| = k$
 - Se $V(< \varphi_{R_\Pi}, w >) = \text{ACC}$ allora R_Π va in loop **volontariamente**.
 - Se $V(< \neg\varphi_{R_\Pi}, w >) = \text{ACC}$ allora R_Π termina.

Claim - Se φ_{R_Π} o $\neg\varphi_{R_\Pi}$ è dimostrabile allora Π è inconsistente.

Nota

La TM è costruita appositamente per fare il contrario dell'affermazione. Notiamo infatti che se troviamo una dimostrazione per "la macchina termina" allora la macchina va in loop volontariamente. Se invece si trova una dimostrazione per "la macchina non termina" la macchina si spegne volontariamente.

Dimostrazione Claim - Supponiamo che $\neg\varphi_{R_\Pi}$ sia dimostrabile. Allora esiste una dimostrazione w t.c. $V(< \neg\varphi_{R_\Pi}, w >) = \text{ACC}$ ovvero $R_{\Pi(< R_\Pi >)}$ termina. Poichè l'esecuzione termina il comportamento della traccia è noto quindi esiste una dimostrazione che descrive la traccia di esecuzione di R_Π che è anche una dimostrazione per φ_{R_Π} . Ha infatti dimostrato che $R_{\Pi(< R_\Pi >)}$ termina ovvero φ_{R_Π} .

Supponiamo adesso che φ_{R_Π} sia dimostrabile. Allora esiste una dimostrazione w t.c. $V_{\Pi}(\varphi_{R_\Pi}, w)$ accetti, ovvero $R_{\Pi(< R_\Pi >)}$ va in loop volontariamente. Poichè va in loop volontariamente il comportamento della traccia è noto e dunque esiste una dimostrazione che descrive la traccia di esecuzione di R_Π , ovvero $R_{\Pi(< R_\Pi >)}$ non termina che sarebbe $\neg\varphi_{R_\Pi}$, che è anche una dimostrazione per $\neg\varphi_{R_\Pi}$.

Quindi, dato che φ_{R_Π} dimostrabile implica che lo sia anche $\neg\varphi_{R_\Pi}$ e viceversa, Π è inconsistente.

Fine dimostrazione teorema - Adesso tornando al teorema, supponiamo Π consistente, ma dobbiamo necessariamente avere per il claim né φ_{R_Π} né $\neg\varphi_{R_\Pi}$ dimostrabili, ovvero Π incompleto.

2.7.2. Secondo Teorema di Incompletezza

Sia Π un sistema di dimostrazione abbastanza potente da comprendere l'aritmetica. Se Π è consistente esso non può dimostrare l'affermazione “ Π è consistente”.

Dimostrazione

- Sia R_Π la TM definita tramite φ_Π come per il teorema precedente
- Valga lo stesso claim del teorema precedente e la sua dimostrazione
- Assumiamo Π consistente, quindi né φ_{R_Π} né $\neg\varphi_{R_\Pi}$ dimostrabili in Π .

Sappiamo dal primo teorema che se Π è consistente allora la macchina R_Π non troverà mai una soluzione per nessuna delle due affermazioni φ_{R_Π} e $\neg\varphi_{R_\Pi}$, ma se non trova una soluzione significa che va in loop, quindi $\neg\varphi_{R_\Pi}$ è vera. Essendo Π sufficientemente potente da comprendere l'aritmetica, questa stessa implicazione è un teorema dimostrabile:

$$\Pi \vdash \text{Cons}(\Pi) \Rightarrow \neg\varphi_{R_\Pi}$$

Letto: *Il sistema Π è in grado di dimostrare il teorema: Se Π consistente allora la macchina R_Π non si fermerà mai ($\neg\varphi_{R_\Pi}$)*

Ipotizziamo per assurdo che Π sia in grado di dimostrare la sua consistenza e quindi:

$$\Pi \vdash \text{Cons}(\Pi)$$

Unendo le due regole otteniamo che Π dimostra $\neg\varphi_{R_\Pi}$, ma per il primo teorema abbiamo dimostrato che se un sistema la dimostra è inconsistente, questa contraddizione fa crollare l'ipotesi e quindi Π non può dimostrare $\text{Cons}(\Pi)$.

In generale è importante ricordare che per dimostrare la consistenza di un sistema matematico è necessario utilizzare un sistema ancora più potente, si crea quindi una successione dove non raggiungeremo mai la massima sicurezza.

3. Complessità

In questa sezione del corso ci si concentra sull'efficienza degli algoritmi (ovvero le TM) nella risoluzione dei problemi, ovvero la decisione dei linguaggi. L'efficienza di un algoritmo può essere espressa in termini di:

- Tempo e Spazio
- Randomness
- Numero di processori per il calcolo parallelo

Definizione - Complessità di Tempo

Sia M una TM decisore, la sua **complessità di tempo** è

$$T : \mathbb{N} \rightarrow \mathbb{N} \text{ t.c. } T(n) = \max_{x \in \Sigma^*, |x|=n} \{\# \text{ passi richiesti da } M(x)\}$$

Ovvero $T(n)$ sia il massimo numero di passi necessari a M per processare una stringa lunga n

Siamo interessati a come una funzione si comporta in termini della lunghezza di x .

Esempio

- La TM M che può fare $\{L, R, S\}$ si può simulare con una TM M' classica e la complessità di tempo sarà $\leq 2T(n) = O(T(n))$.
- La TM M con k nastri si può simulare con una TM M' classica e la complessità di tempo sarà $O(T(n)^2)$. Questo perchè costruire il nastro richiede $O(T(n))$ mentre per la simulazione abbiamo diversi casi:
 - Per simulare un passo si deve scorrere tutto il nastro e in più si potrebbe dover traslare tutto il nastro a destra di una posizione. Il nastro sarà lungo al più $O(n)$ quindi con n passi abbiamo $O(T(n)^2)$. In totale quindi

$$O(T(n)) + O(T(n)^2)$$

Osservazione - Ogni TM a singolo nastro necessita di $\Omega(n^2)$ tempo per decidere PALINDROMES. Se si usassero due nastri invece $O(n)$.

Definizione - D-Time

Sia $t : \mathbb{N} \rightarrow \mathbb{R}^+$. Definiamo

$$\text{DTIME}(t(n)) = \{L : \exists \text{ TM che decide } L \text{ in tempo } O(t(n))\}$$

Quindi ad esempio, $\text{PALINDROMES} \in \text{DTIME}(n^2)$ e invece $\notin \text{DTIME}(n)$

Definizione - Classe P

P è la classe di linguaggi decidibili da una TM in tempo polinomiale.

$$P = \bigcup_{k \in \mathbb{N}} \text{DTIME}(n^k)$$

Teorema

$$\text{PATH} = \{\langle G, s, t \rangle \mid \exists s \rightsquigarrow t \in G\} \in P$$

Dove G è un grafo diretto e vogliamo trovare un percorso da s a t .

Dimostrazione - Vogliamo dimostrare che esiste una TM che in tempo polinomiale ci può dire se esiste un percorso in grafo diretto G che ci porta da s a t .

Costruiamo la TM $M = \text{"Data } \langle G, s, t \rangle$

- Marca il vertice s
- Finché non vengono marcati nuovi vertici
 - Marca ogni vertice con un arco entrante da un vertice già marcato
- Se t è marcato, accetta; altrimenti rifiuta"

Definiamo $n = |V|$ il numero di vertici e $m = |E|$ il numero di archi. Nel caso peggiore in assoluto ad ogni ciclo marcheremo un solo vertice, siccome ci sono n vertici il ciclo può ripetersi massimo n volte. Inoltre ad ogni ciclo dobbiamo guardare tutte le frecce collegate, scorrerle tutte costa m . Il tempo totale è quindi $O(m \cdot n)$.

In un grafo orientato, nel peggiore dei casi, abbiamo un grafo completamente connesso dove ogni nodo ha una freccia che lo collega ad ogni altro nodo, quindi gli archi massimi crescono fino a circa

n^2 . La complessità di tempo è quindi $T(n) = O(n \cdot n^2) = O(n^3)$. La complessità è quindi polinomiale.

Teorema

$$2\text{COL} = \{\langle G \rangle \mid G \text{ è 2-colorabile}\} \in P$$

Dimostrazione - Il numero delle possibili colorazione è $O(2^n) \in \text{EXP}$ però per capire se è 2-colorabile possiamo usare il seguente algoritmo:

- Prendo un vertice e lo coloro di BLU
- Coloro i suoi vicini di ROSSO
- Coloro i vicini dei vicini di BLU
- ...
- Se trovo contraddizioni, rifiuto; altrimenti ripeto per tutte le componenti connesse di G . Se ho colorato tutto, accetto.

L'algoritmo esplora tutto il grafo una sola volta quindi ha complessità $O(n)$.

Definizione - Classe EXP

EXP è la classe dei linguaggi decidibili da una TM in tempo esponenziale

$$\text{EXP} = \bigcup_{k \in \mathbb{N}} \text{DTIME}(2^{n^k})$$

Time Hierarchy Theorem

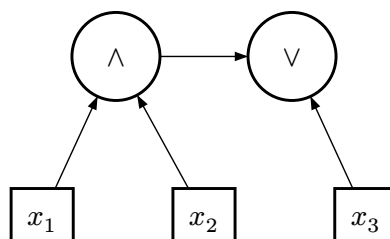
$$\exists L \text{ t.c. } L \in \text{EXP} \text{ ma } L \notin P, \text{ ovvero } P \neq \text{EXP}$$

3.1. Satisfiability

La soddisfacibilità è uno dei problemi fondamentali della teoria della complessità, consiste nel determina se una formula booleana è soddisfacibile.

Definizione - Circuito Booleano

Un circuito booleano C è un grafo aciclico con n input $x_1, \dots, x_n$ e un output. I vertici sono detti **porte** ($\wedge, \vee, \neg$). Il **fan-out** di una porta è il numero di archi uscenti da essa.



In questo caso la porta $\wedge$ ha fan-out 1 mentre la porta $\vee$ ha fan-out 0. Inoltre abbiamo $F : (x_1 \wedge x_2) \vee x_3$

Un circuito booleano con fan-out massimo 1 è definito **formula** booleana.

Definizione - Circuit-eval

$$\text{Circuit-eval} = \{\langle C, x \rangle \text{ con } C \text{ circuito booleano e } C(x) = 1\}$$

Quindi riceviamo un circuito ed un input, la coppia appartiene al linguaggio soltanto se il circuito produce 1 come output.

Osservazione - Sia C con n input n porte, $|\langle C \rangle| = O(n \log n)$ e quindi $\text{circuit-eval} \in P$. Il costo è giustificato dal fatto che il circuito ha n porte e per descrivere ciascuna di esse in binario abbiamo bisogno di $\log(n)$ bit.

Definizione - Circuit-SAT

$$\text{Circuit-SAT} = \{\langle C \rangle \mid \exists x \in \{0, 1\}^n \text{ t.c. } C(x) = 1\}$$

Osservazione - $\text{Circuit-SAT} \in \text{EXP}$ perché posso deciderlo in $O(|\langle C \rangle| \cdot 2^n)$

Esistono alcune varianti:

- **FORMULA-SAT**: C è una formula
- **CNF-SAT**: C è una CNF, ovvero una formula in una forma fatta esclusivamente da una catena di clausole tenute insieme da operatori AND. Una clausola è un gruppo di letterali tenuti insieme da operatori OR. I letterali sono le variabili (anche negate).
- **K-SAT**: CNF-SAT in cui tutte le clausole hanno k letterali

Teorema

$$2\text{-SAT} \in P$$

Dimostrazione - Sia $\varphi(x_1, \dots, x_n)$ la CNF 2SAT con n variabili e n clausole. La rappresentiamo come grafo in questo modo:

- Per ogni clausola $x \vee y$ consideriamo due implicazioni logiche equivalenti:

$$\bar{x} \rightarrow y \text{ e } \bar{y} \rightarrow x$$

- Gli archi del grafo rappresentano queste implicazioni.

Claim - φ è soddisfacibile $\Leftrightarrow$ nessuna componente fortemente connessa di G contiene una variabile e la sua negazione. Una componente fortemente connessa è un insieme di nodi tutti tali che presi due nodi qualsiasi esiste un percorso che li collega.

Dimostrazione Claim - Supponiamo φ soddisfacibile ($\Rightarrow$). Supponiamo per assurdo ci sia una componente fortemente connessa che contiene x e $\bar{x}$. Poiché è una CFC, vuol dire che $\exists x \rightsquigarrow \bar{x}$ e $\bar{x} \rightsquigarrow x$. Ma poiché gli archi sono implicazioni equivale a dire che valgono $x \Rightarrow \bar{x}$ e $\bar{x} \Rightarrow x$, impossibile se $\varphi \in \text{SAT}$, quindi x e $\bar{x}$ non saranno mai nella stessa CFC.

Adesso assumiamo che nessuna CFC contenga sia x che $\bar{x}$ ($\Leftarrow$). Definiamo un ordinamento topologico delle componenti e il seguente assegnamento:

$$\alpha(x) = \begin{cases} \text{True} & \text{se } \text{ord}(x) > \text{ord}(\neg x) \\ \text{False} & \text{se } \text{ord}(x) < \text{ord}(\neg x) \end{cases}$$

Dimostriamo che l'assegnamento non viola implicazioni: Supponiamo per assurdo che $\exists(U, V)$ t.c. $U = \text{true}$ e $V = \text{false}$.

Dato che esiste l'arco $u \rightarrow v$, l'ordinamento topologico deve rispettare la direzione dell'arco ovvero $\text{ord}(U) \leq \text{ord}(V)$. Dato che siamo in 2-SAT deve anche esistere l'arco $(\bar{V}, \bar{U})$ per cui $\text{ord}(\bar{V}) \leq \text{ord}(\bar{U})$.

Per la supposizione fatta $U = \text{True}$ abbiamo che $\text{ord}(\bar{U}) < \text{ord}(U)$, sempre per la supposizione abbiamo $V = \text{False}$ e abbiamo quindi che $\text{ord}(V) < \text{ord}(\bar{V})$.

Mettendo tutto insieme otteniamo:

$$\text{ord}(V) < \text{ord}(\bar{V}) \leq \text{ord}(\bar{U}) < \text{ord}(U) \leq \text{ord}(V)$$

Leggendo i termini agli estremi abbiamo che $\text{ord}(V) < \text{ord}(V)$ che è un assurdo. L'assegnamento soddisfa quindi φ .

Per quanto riguarda la complessità abbiamo la costruzione del grafo $O(n + m)$, l'algoritmo per trovare le componenti $O(n + m)$ e la verifica delle componenti $O(n)$, essendo tutto lineare $2\text{SAT} \in P$.

Definizione - Verificatore

Una TM V è un verificatore per un linguaggio L se:

- V ha input $\langle x, y \rangle$ con $x \in \Sigma^*$ e y certificato (configurazione da verificare)
- $\forall x, x \in L \Leftrightarrow \exists y \text{ t.c. } V(\langle x, y \rangle) = \text{ACC}$

Un verificatore può portare a due casi:

- YES CASE: $x \in L \Rightarrow \exists y \text{ t.c. } V(\langle x, y \rangle) = \text{ACC}$
- NO CASE: $x \notin L \Rightarrow \forall y, V(\langle x, y \rangle) = \text{REJ}$

È importante notare che un verificatore V ha tempo di esecuzione polinomiale se il suo tempo di esecuzione è $O(|x|^k)$ per $k \in \mathbb{N}$. Come conseguenza y deve essere polinomiale in n : $|y| = \text{poly}(n)$ perchè V deve leggere y , altrimenti il tempo di lettura supererebbe quello di esecuzione. La regola fondamentale è quindi che il verificatore deve lavorare in tempo polinomiale rispetto alla grandezza del problema originale.

Definizione - VERIFIER-NP

Possiamo descrivere la classe NP come l'insieme dei linguaggi L t.c. L ha un verificatore in tempo polinomiale.

Teorema

$3\text{COL} \in \text{NP}$

Dimostrazione - Consideriamo il seguente verificatore V :

- Su input $\langle x, y \rangle$ dove x è un grafo $G = (V', E)$.
 - Interpreta $y = \langle c_1, \dots, c_n \rangle$ dove $n = |V'|$ e $c_i \in \{R, Y, B\}$. È quindi una lista che ci indica per ogni nodo come è colorato.
 - $\forall (i, j) \in E, \text{REJECT} \Leftrightarrow c_i = c_j$
 - Se dopo aver controllato tutti gli archi non ha mai rifiutato, "ACCEPT"

V ha tempo polinomiale in $|x|$ dato che controlla una sola volta tutto il grafo.

Inoltre:

- YES CASE: $G \in 3COL \Rightarrow \exists 3COL \ y = (c_1, \dots, c_n)$ t.c. $\forall (i, j) \in E, c_i \neq c_j$ allora $V(\langle x, y \rangle) = ACC$. Ovvero se forniamo una soluzione al verificatore ci dá la risposta corretta.
- NO CASE: Se $V(\langle x, y \rangle) = ACC$ per qualche y allora y é una 3COL del grafo e quindi $G \in 3COL$ perchè ha controllato tutti gli archi. Non ci sono falsi positivi ovvero che diamo un certificato non valido ma il verificatore ci dice che lo é.

Teorema

$$P \subseteq NP \subseteq EXP$$

Per il Time Hierarchy Theorem dobbiamo avere $P \neq NP$ oppure $NP \neq EXP$ oppure entrambe.

Dimostrazione - Prima dimostriamo $P \subseteq NP$. Sia $L \in P$ vuole dire che $\exists M$ t.c. $L(M) = L$ e M ha tempo polinomiale. Per vedere che $L \in NP$ basta considerare il verificatore che su input $\langle x, y \rangle$ ignora y e accetta se e solo se $M(x) = ACC$.

Dimostriamo adesso che $NP \subseteq EXP$. Sia $L \in NP$ allora esiste un verificatore V in tempo polinomiale e il certificato è lungo al massimo $|x|^k$ con $k \in \mathbb{N}$. Esiste allora una TM M che genera tutte le possibili stringhe y di lunghezza $|x|^k$ e per ognuna esegue il verificatore V . Se V accetta anche solo una volta allora M accetta altrimenti rifiuta. Dato che ci sono in totale circa $2^{(|x|^k)}$ combinazioni di y che richiedono tempo polinomiale per essere controllate allora il tempo totale diventa esponenziale quindi M lavora in EXP.

Possiamo definire NP anche in termini di macchine di Turing non-deterministiche (Non-deterministic Polynomial time).

Definizioni

$$NTIME(f(n)) = \{L \text{ t.c. } \exists \text{ NTM } N \mid L(N) = L \text{ e } N \text{ ha tempo } O(f(n))\}$$

$$NP = \bigcup_{k \in \mathbb{N}} NTIME(n^k)$$

$$NEXP = \bigcup_{k \in \mathbb{N}} NTIME(2^{n^k})$$

Teorema

$$3SAT \in NP$$

Dimostrazione - Prendiamo la NTM N t.c. su input $\langle x \rangle = \langle \varphi \rangle$ 3SAT formula: Prova non deterministicamente tutti i possibili assegnamenti $x_1, \dots, x_n$ e controlla $\varphi(x_1, \dots, x_n)$.

Teorema

$$VERIFIER-NP = NP$$

Mostriamo che le due definizioni sono equivalenti, una quella del verificatore con il certificato e l'altra quella della TM non deterministica.

Dimostrazione - ($\Rightarrow$) Supponiamo che il problema L abbia un verificatore V che lavora in tempo polinomiale. Vogliamo costruire una NTM N che risolva lo stesso problema. Dato che V lavora in tempo polinomiale allora il certificato y deve essere $|y| = O(|x|^k)$.

La NTM quindi: Su input x la macchina N genera tutte i certificati y possibili di lunghezza polinomiale, e accetta quando un ramo eseguendo $V(\langle x, y \rangle)$ accetta.

La NTM N quindi accetta x se e solo se un suo ramo ha generato un certificato y valido.

($\Leftarrow$) - Supponiamo adesso di avere una NTM N che risolve il problema in modo non deterministico in tempo polinomiale. Vogliamo costruire un verificatore deterministico V di tempo polinomiale. Il certificato sarà l'insieme di scelte non-deterministiche di N rappresentate come stringa di lunghezza polinomiale. Ora costruiamo V che prende in input il problema x e il certificato y , simula la macchina N ma non si sdoppia mai dato che in y sono contenute le scelte corrette.

V accetterà se e solo se la mappa y lo porta di uno stato di accettazione di N e se x appartiene al linguaggio quel percorso esiste perché N accetta e quindi esisterà un certificato y .

$$x \in L \Leftrightarrow N \text{ accetta} \Leftrightarrow \exists y \text{ insieme di scelte per cui } N \text{ accetta} \Leftrightarrow \exists y \text{ t.c. } V(\langle x, y \rangle) = \text{ACC}$$

Definizione - Riduzione

Siano A, B linguaggi:

$$A \leq_m^p B \text{ se } \exists \text{ poly-time } R : \Sigma^* \rightarrow \Sigma^* \text{ t.c. } \forall x \in \Sigma^* : x \in A \Leftrightarrow R(x) \in B$$

Quindi come le mapping reduction ma R deve essere calcolabile in tempo polinomiale.

Teorema

Se $A \leq_m^p B$ e $B \in P$, allora $A \in P$

Dimostrazione - Sia M_B l'algoritmo poly-time che decide B costruisco la TM $M_{A(x)} = M_{B(R(x))}$ dove R è la riduzione $A \leq_m^p B$ e quindi M_A è polinomiale. Infatti prende l'input x lo passa alla funzione R e poi il risultato lo passa ad M_B , entrambe sono polinomiali e quindi M_A è polinomiale. Formalmente $x \in A \Leftrightarrow R(x) \in B \Leftrightarrow M_{B(R(x))} = \text{ACC} \Leftrightarrow M_{A(x)} = \text{ACC}$

Teorema

Se $A \leq_m^p B$ e $B \in NP$ allora $A \in NP$

Dimostrazione - La dimostrazione è analoga alla precedente ma si sostituiscono M_A e M_B con NTM N_A e N_B

Osservazione - La riduzione $\leq_m^p$ è transitiva. Se $A \leq_m^p B$ e $B \leq_m^p C$ allora $A \leq_m^p C$. Inoltre osserviamo anche che $X \leq_m^p Y \Rightarrow \overline{X} \leq_m^p \overline{Y}$

Teorema

$$4\text{COL} \leq_m^p \text{SAT}$$

Dimostrazione - Dato un grafo $G = (V, E)$ 4-colorabile devo costruire una formula φ_G t.c. $\varphi_G \in \text{SAT} \Leftrightarrow G \in 4\text{COL}$.

La formula avrà $2n$ variabili $x_1, x_{1'}, \dots, x_n, x_{n'}$ dove $n = |V|$ e ogni coppia di variabili $x_i, x_j \in \{0, 1\}$ rappresenta una codifica di un colore.

Nella formula, per ogni arco $(i, j) \in E$ codifico la regola "avere colori diversi" nel seguente modo: $(x_i, x_{i'}) \neq (x_j, x_{j'})$ ovvero $\neg(x_i \Leftrightarrow x_j \wedge x_{i'} \Leftrightarrow x_{j'})$. Traducendo ogni $x_i \rightarrow x_j$ con $\neg x_i \vee x_j$ ottengo una formula φ_{ij} . Ottengo quindi $\varphi_{G(\cdot)} = \bigwedge_{(i,j) \in E} \varphi_{i,j}(\cdot)$

La formula è corretta quindi $\varphi_G \in \text{SAT} \Leftrightarrow G \in 4\text{COL}$

Teorema

$$3\text{COL} \stackrel{p}{\leq}_m 4\text{COL}$$

Dimostrazione - Devo definire $R(\langle G \rangle) = \langle H \rangle$ t.c. $G \in 3\text{COL} \Leftrightarrow H \in 4\text{COL}$.

È molto semplice, infatti se G è 3 colorabile aggiungo un nodo e lo collego a tutti i nodi di G , questo nuovo grafo H avrà il nuovo nodo che posso colorare con un nuovo colore e diventerà quindi 4 colorabile.

Osservazione - Usando la transitività di prima osserviamo che $3\text{COL} \stackrel{p}{\leq}_m 4\text{COL} \stackrel{p}{\leq}_m \text{SAT} \Rightarrow 3\text{COL} \stackrel{p}{\leq}_m \text{SAT}$

Definizione - NP-HARD

Un linguaggio S è NP-HARD se $\forall L \in NP, L \stackrel{p}{\leq}_m S$

Definizione - NP-COMPLETEZZA

Un linguaggio S è NP-COMPLETO se S è NP-HARD e $S \in NP$.

Teorema

Se S è NP-COMPLETO allora $S \in P \Leftrightarrow P = NP$

Dimostrazione - Se S è NP-COMPLETO allora $\forall L \in NP$ abbiamo $L \stackrel{p}{\leq}_m S$. Se $S \in P$ allora ogni $L \in P$ e quindi $P = NP$.

L'altra implicazione invece se $P = NP$ e S NP-COMPETO abbiamo che $S \in NP = P$ ovvero $S \in P$

Teorema - Cook-Levin

$$\text{SAT} \in \text{NP-COMPLETE}$$

Dimostrazione - Dimostriamo che $\text{SAT} \in \text{NP-HARD}$ ovvero che dato un qualsiasi $L \in NP$ allora $L \stackrel{p}{\leq}_m \text{SAT}$.

Dato un qualsiasi problema $L \in NP$ sappiamo che esiste una NTM N che lo risolve in tempo polinomiale. Creiamo un tableau tale che:

- Le righe rappresentano i passi di computazione
- Le colonne rappresentano le celle del nastro della TM

- In ogni istante (riga) il nastro contiene dei simboli e la testina della macchina si trova in una certa posizione in un certo stato.

Il nostro obiettivo é creare una formula φ_N che sia soddisfacibile se e solo se esiste un modo di riempire questa griglia in modo che rappresenti un'esecuzione accettante della macchina N .

Per trasformare questa griglia in logica, inventiamo una variabile booleana $x_{i,j,s} = 1 \Leftrightarrow$ nella riga i alla colonna j c'è il simbolo s .

La grande formula φ_N é un insieme di 4 AND fra formule che impongono delle regole al tableau affinché si comporti come una vera TM.

METERE FOTO CARINA

Costruisco la formula

$$\varphi_N = \varphi_{\text{cell}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{move}} \wedge \varphi_{\text{accept}}$$

Dove:

- φ_{cell} : Ogni cella contiene un solo simbolo

$$\varphi_{\text{cell}}(x) = \bigwedge_{i,j \in [n^k]} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{s,t \in C, s \neq t} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right]$$

Ovvero per ogni cella, ogni cella deve avere almeno un valore e non ne può avere 2.

- φ_{start} : La prima riga deve contenere la configurazione iniziale $\#q_0w_1...w_n\# \sqcup \dots$

$$\varphi_{\text{start}}(x) = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge \dots \wedge x_{1,n^k,\#}$$

- φ_{accept} : Esiste una configurazione accettante

$$\varphi_{\text{accept}}(x) = \bigvee_{i,j \in [n^k]} x_{i,j,q_{\text{acc}}}$$

- φ_{move} : Ogni riga del tableau é consistente rispetto a δ_N e alla riga precedente, usiamo la finestra 2x3, una finestra é lecita se non viola le regole di δ_N . Questa regola controlla appunto che il tableau segua le regole della TM, la finestra 2x3 é la piú piccola che ci serve infatti una cella nella riga $i + 1$ la possiamo verificare semplicemente guardando la cella alla riga i e le due celle adiacenti ad essa. Se tutte le finestre possibili sono legali allora l'intera azione é legale.

Osservazione - Se la prima riga é la configurazione iniziale e ogni finestra del tableau é lecita, allora ogni riga segue la precedente secondo δ_N

$$\varphi_{\text{move}}(x) = \bigwedge_{i,j \in [n^k]} (\text{finestra } (i,j) \text{ lecita})$$

Il tableau ha $O(n^k \cdot n^k) = O(n^{2k})$ celle quindi $O(n^{2k})$ variabili. Quindi φ_{cell} , φ_{move} e φ_{acc} hanno dimensione $O(n^{2k})$. Dunque φ_N ha dimensione polinomiale. Perciò la formula può essere generata in tempo polinomiale ed é semplice costruire una riduzione.

Osservazione - Per dimostrare che un problema é NP-Completo si procede tipicamente per riduzione a SAT. Infatti abbiamo mostrato che possiamo prendere una qualsiasi TM (problema) e trasformarlo in una formula SAT.

Teorema

K-CLIQUE (CLIQUE) = $\{ \langle G, k \rangle \mid G \text{ é un grafo e contiene una K-clique} \}$ é NP-COMPLETO.

Una k-clique é un gruppo di k vertici in cui ogni coppia di vertici é collegata da un arco.

Dimostrazione - Facciamo una riduzione $3SAT \stackrel{p}{\leq}_m CLIQUE$. L'obiettivo è quello di trasformare una formula logica in un grafo

Prendiamo quindi una formula 3SAT formata da k parentesi (clausole) ognuna con 3 letterali, sia $\varphi = (a_1 \vee b_1 \vee c_1) \wedge \dots \wedge (a_k \vee b_k \vee c_k)$. Vogliamo costruire un grafo in cui se troviamo una "clique" di grandezza k significa che abbiamo trovato la soluzione alla formula. $\langle G, k \rangle$ t.c. $\varphi \in 3SAT \Leftrightarrow \langle G, k \rangle \in CLIQUE$

Per ogni letterale di ogni parentesi disegniamo un nodo, inseriamo archi fra tutti i nodi ad eccezione di due casi:

- Nessun arco fra nodi della stessa clausola
- Nessun arco deve formare una contraddizione, quindi non colleghiamo mai x_i con $\overline{x_i}$

($\Rightarrow$) - Se la formula è soddisfatta allora c'è la clique: Se la formula φ è vera significa che abbiamo trovato una combinazione logica che accende almeno un letterale in ogni parentesi. Coloriamo nel grafo i nodi corrispondenti a questi letterali e siccome appartengono a clausole diverse, per costruzione, saranno collegati.

($\Leftarrow$) - Se c'è la clique, allora la formula è soddisfatta: Supponiamo quindi di aver trovato una clique nel grafo, dato che non ci sono archi fra nodi della stessa clausola l'unico modo per avere k nodi connessi è averne preso uno per ogni clausola, inoltre non ci sono contraddizioni sempre perché sono tutti connessi. Accendendo i letterali corrispondenti abbiamo trovato una formula valida.

Costruire il grafo richiede tempo polinomiale e anche risolvere la clique nel grafo equivale esattamente a risolvere la formula 3SAT, abbiamo dimostrato che CLIQUE è NP-COMPLETO.

Adesso ci chiediamo se esiste un linguaggio $L \in NP$ e $L \notin P$ ma non NP-Completo.

Teorema di Ladner

Esistono linguaggi $\notin P$ ma $\in NP$ e non NP-Completi

Ovvero linguaggi che non sappiamo risolvere velocemente ma che non sono abbastanza "potenti" da fare da chiave per la riduzione per tutta la classe NP.

Definizione

$$coNP = \{L \mid \overline{L} \in NP\}$$

Fino ad adesso abbiamo visto che i linguaggi in NP sono facili da verificare, ovvero se la risposta al problema è "SI" esiste un certificato veloce da verificare. coNP è l'opposto, se la risposta è "NO" allora esiste un certificato veloce da verificare.

Osserviamo che $coNP \neq \overline{NP}$

Teorema

$$SAT \in P \Leftrightarrow UNSAT \in P$$

Dimostrazione - Se $SAT \in P$ allora esiste una TM di tempo polinomiale che data $\langle \varphi \rangle$ accetta $\Leftrightarrow \varphi \in SAT$. Per decidere UNSAT scambiamo ACCEPT con REJ.

Teorema

P é chiusa per complemento:

$$L \in P \Leftrightarrow \bar{L} \in P \quad (P = \text{co}P)$$

Teorema

$$\text{EXP} = \text{coEXP}$$

Questi teoremi ci dicono essenzialmente che la classe P e $\text{co}P$ sono la stessa cosa, infatti se un linguaggio appartiene ad una delle due ed esiste quindi una TM deterministica in tempo polinomiale, ci basta invertire gli output di questa.

Teorema

$$\text{coNP} \subseteq \text{EXP}$$

Dimostrazione - Sia $L \in \text{coNP}$, allora abbiamo che $\bar{L} \in \text{NP} \subseteq \text{EXP}$ questo significa che $L \in \text{coEXP} = \text{EXP}$

Teorema

$$P \subseteq \text{coNP}$$

Dimostrazione - Prendiamo un problema $L \in P$ e visto che P é chiuso per complemento abbiamo $\bar{L} \in P$. Sappiamo che $P \subseteq \text{NP}$ e quindi $\bar{L} \in \text{NP}$ ma allora $L \in \text{coNP}$.

Teorema

$$P = \text{NP} \Rightarrow P = \text{coNP}$$

Dimostrazione - Se $L \in \text{coNP}$ allora $\bar{L} \in \text{NP}$ ma dato che $P = \text{NP}$ il suo inverso sta anche in P . Ma se invertiamo le risposte abbiamo quindi che anche $L \in P$. Quindi se NP collapsasse dentro P allora si porterebbe con se anche coNP .

Corollario - Basandoci sul teorema precedente allora possiamo anche dire che $\text{NP} \neq \text{coNP} \Rightarrow P \neq \text{NP}$.

Teorema

$$\text{NP} = \text{coNP} \Leftrightarrow \text{UNSAT} \in \text{NP}$$

Dimostrazione - Prima dimostriamo $(\Rightarrow)$, quindi sia $\text{NP} = \text{coNP}$ allora $\text{UNSAT} \in \text{coNP}$ che é uguale a NP .

$(\Leftarrow)$ - Sia $\text{UNSAT} \in \text{NP}$. Prendiamo $L \in \text{coNP}$ quindi $\bar{L} \in \text{NP}$. Per NP-Completezza, abbiamo $\bar{L} \stackrel{p}{\leq}_m \text{UNSAT}$ e quindi $\bar{L} \stackrel{p}{\leq}_m \text{SAT} \Leftrightarrow L \stackrel{p}{\leq}_m \text{UNSAT}$. Ma $\text{UNSAT} \in \text{NP}$ quindi $L \in \text{NP}$.

Definizione - coNP-Completezza

Un linguaggio L è coNP-Completo se:

- $L \in \text{coNP}$
- $\forall A \in \text{coNP}, A \leq_m^p L$

Teorema

“UNSAT” è NP-Completo

Dimostrazione - Sappiamo che $\text{UNSAT} \in \text{coNP}$. Inoltre $A \leq_m^p \text{UNSAT} \Leftrightarrow \overline{A} \leq_m^p \text{SAT}$. Visto che SAT è NP-Completo allora $\overline{A} \leq_m^p \text{SAT}$.

Quindi visto che $\overline{A} \leq_m^p \text{SAT}$ si ha anche $A \leq_m^p \text{UNSAT}$.

Quindi abbiamo che:

- Se $L \in \text{NP}$ allora esiste un verificatore in tempo polinomiale $V(x, y)$ tale che $\forall x, x \in L \Leftrightarrow \exists y \text{ t.c. } V(x, y) = \text{ACC}$
- Se $L \in \text{coNP}$ allora esiste un verificatore in tempo polinomiale $V(x, y)$ tale che $\forall x, x \notin L \Leftrightarrow \exists y \text{ t.c. } V(x, y) = \text{ACC}$

3.2. Space Complexity

Vogliamo misurare l'efficienza delle TM in termini di spazio.

Definizione - Complessità di Spazio

La complessità di spazio di un decisore M è una funzione

$$S : \mathbb{N} \rightarrow \mathbb{N} \text{ t.c. } S(n) = \max_{x, |x|=n} \{\# \text{celle del nastro scritte da } M(x)\}$$

Dato che l'input ha dimensione n non vogliamo essere penalizzati dalla sua lettura, cambiamo quindi il modello di TM.

M ha due nastri, uno di input in sola lettura e quello di output che sarà quello di lavoro.

Consideriamo ad esempio la TM multinastro, per la complessità di tempo passare da una multinastro ad una singola genera un overhead da $T(n)$ a $O(t(n)^2 z)$. Per lo spazio invece soltanto da $S(n)$ a $O(S(n))$.

Definizione

$$\text{SPACE}(f(n)) = \{L \mid \exists \text{ TM } M \text{ con complessità di spazio } O(f(n)) \text{ t.c. } L(M) = L\}$$

Definizione

$$\text{NSPACE}(f(n)) = \{L \mid \exists \text{ NTM } N \text{ con complessità di spazio } O(f(n)) \text{ t.c. } L(N) = L\}$$

Osservazioni:

- Per quanto riguarda il tempo, la complessità è almeno lineare in n dato che la TM deve leggere l'input.

- Per lo spazio invece la complessità è almeno $\log n$, possiamo “ignorare” l’input e considerare soltanto la memoria necessaria a far funzionare l’algoritmo con una variabile che ricorda a che punto dell’input siamo, per contenerlo tutto serve quindi almeno $\log n$ spazio.

Per la complessità di spazio le principali classi sono:

- $L = \text{SPACE}(\log n)$ e $NL = \text{NSPACE}(\log n)$. Quindi i problemi risolvibili usando una memoria proporzionale al logaritmo dell’input.
- $\text{PSPACE} = \cup_k \text{SPACE}(n^k)$ e $\text{NSPACE} = \cup_k \text{NSPACE}(n^k)$. Problemi risolvibili con una memoria polinomiale rispetto all’input.
- $\text{EXPSPACE} = \cup_k \text{SPACE}(2^{n^k})$ e $\text{NEXPSPACE} = \cup_k \text{NSPACE}(2^{n^k})$

Mettendo tutto insieme con anche le complessità di tempo abbiamo:

$$P \subseteq NP \subseteq \text{PSPACE} \subseteq \text{EXP} \subseteq \text{NEXP} \subseteq \text{EXPSPACE}$$

- $N \subseteq NP$: Lo sapevamo da prima
- $NP \subseteq \text{PSPACE}$: Un problema risolvibile in modo non deterministico in tempo polinomiale possiamo risolverlo con memoria polinomiale, infatti basta esplorare un ramo alla volta e tenere in memoria soltanto quel ramo.
- $\text{PSPACE} \subseteq \text{EXP}$: Se una TM ha a disposizione uno spazio polinomiale, quante configurazioni diverse può assumere il nastro prima di ripetersi? Un numero esponenziale, quindi se la macchina lavorasse in tempo esponenziale finirebbe in un loop.
- $\text{EXP} \subseteq \text{NEXP}$: Come P ed NP ma in tempo esponenziale.
- $\text{NEXP} \subseteq \text{EXPSPACE}$: Stesso funzionamento del punto 2 ma usiamo ordini di grandezza esponenziali.

Teorema

$$\text{TIME}(f(n)) \subseteq \text{SPACE}(f(n))$$

Dimostrazione - Una TM con tempo $O(f(n))$ può scrivere al più $O(f(n))$ celle.

Corollario - $P \subseteq \text{PSPACE}$, $NP \subseteq \text{NPSPACE}$

Teorema

$$NP \subseteq \text{PSPACE}$$

Dimostrazione - Segue dal teorema di Savitch (si vede dopo) ma si può dimostrare anche direttamente.

Sappiamo che $A \in NP$ se esiste una TM in tempo polinomiale che $\forall x, x \in L \Leftrightarrow \exists y$ t.c. $V(x, y) = 1$. Siccome V è in tempo polinomiale allora $|y| = p(n)$ per un polinomio p .

Per decidere A in spazio polinomiale:

- Memorizzo il candidato y sul nastro di lavoro 1.
- Simulo $V(x, y)$ sul nastro 2.
- Se $V(x, y)$ accetta, accetto; altrimenti passo all’ y successivo riutilizzando lo stesso spazio.

Teorema - Lo spazio limita il tempo

Per ogni $f(n) \geq \log n$, $\text{SPACE}(f(n)) \subseteq \text{DTIME}(2^{O(f(n))})$

Quello che ci dice il teorema è che se un algoritmo usa una memoria pari a $f(n)$ allora il tempo massimo che impiegherà per arrivare a una soluzione sarà proporzionale a un'esponenziale di quello spazio ovvero $2^{O(f(n))}$

Dimostrazione - La dimostrazione si basa sul "fotografare" la TM durante l'esecuzione e vedere quante configurazioni possibili ci sono, se la macchina si trova due volte nella stessa configurazione significa che è andata in loop ma stiamo parlando di un decisore quindi non accadere.

Quindi abbiamo che il tempo di esecuzione deve essere minore o uguale al numero totale di configurazioni possibili. Ma quante sono?

- $|Q|$: Stati della TM
- $f(n)$: Posizione della testina, dato che il nastro è lungo al massimo $f(n)$ la testina può trovarsi in una di queste celle.
- n : Nastro di input, anche qui la testina può trovarsi in una di queste celle, dato che abbiamo imposto che $f(n) \geq \log n$, per le regole dei logaritmi possiamo dire che $n \leq 2^{f(n)}$.
- $|\Gamma|^{f(n)}$: È il nastro di lavoro, abbiamo $f(n)$ celle che possiamo riempire con un alfabeto di $|\Gamma|$ simboli, quindi le combinazioni totali sono $|\Gamma|^{f(n)}$.

Moltiplichiamo tutto fra loro, $|Q|$ e $|\Gamma|$ sono costanti, $f(n)$ è un valore piccolo per una potenza di 2 ($2^{f(n)}$) e per un'altra potenza $|\Gamma|^{f(n)}$, quindi il risultato asintotico è $2^{O(f(n))}$, il valore è limitato dallo spazio utilizzato.

Corollario:

- $L \subseteq P$. Se la complessità di spazio è logaritmica cioè in L , applicando la formula del tempo $2^{O(\log(n))}$, per le proprietà dei logaritmi, l'esponenziale e il logaritmo si annullano e il risultato diventa un polinomio. Quindi se si usa spazio logaritmico si impiega tempo polinomiale.
- $\text{PSPACE} \subseteq \text{EXP}$: Se invece lo spazio è un polinomio e applichiamo la formula per il tempo $2^{O(n^k)}$ otteniamo un tempo esponenziale.

Mettendo insieme tutto quello che abbiamo scoperto

$$L \subseteq P \subseteq \text{PSPACE} \subseteq \text{EXP}$$

Sappiamo che $P \neq \text{EXP}$ e quindi o $P \neq \text{PSPACE}$ o $\text{PSPACE} \neq \text{EXP}$ o entrambi.

Teorema

$$\text{PATH} \in \text{SPACE}(\log^2 n)$$

Dimostrazione - PATH è il problema che ci dice se esiste un percorso tra un nodo di partenza s e un nodo di arrivo t , se il percorso esiste non può essere più lungo di n nodi del grafo. Per comodità arrotondiamo la lunghezza massima alla potenza di 2 più vicina, quindi se ad esempio i nodi sono 100 cerchiamo i percorsi lunghi fino a $2^7 = 128$, lunghezza massima che ci interessa è quindi 2^k dove k è approssimativamente $\log_2(n)$.

Quindi formalmente la funzione PATH? prende in input (x, y, k) e ritorna "SI" $\Leftrightarrow \exists x \rightsquigarrow y$ con lunghezza $\leq 2^k$.

L'idea alla base della funzione é che se esiste questo percorso allora deve esistere un nodo a metà strada w t.c.:

- Possiamo percorrere da x a w nella prima metà di passi cioè 2^{k-1}
- Possiamo arrivare da w a y nella seconda metà cioè 2^{k-1}

La funzione é definita in modo ricorsivo:

- Caso Base ($k = 0$): Se il limite é $2^0 = 1$ passi controlliamo subito se esiste un arco tra x ed y , oppure se $x = y$, in caso positivo accetto.
- Passo Ricorsivo: Come detto prima allora deve esistere un nodo w t.c.:
 - $x \rightsquigarrow w$ in $\leq 2^{k-1}$ passi
 - $w \rightsquigarrow y$ in $\leq 2^{k-1}$ passi

Ovvero chiede prima $\text{PATH?}(x, w, k - 1)$ e poi in $\text{AND PATH?}(w, y, k - 1)$

In ogni chiamata della funzione devo memorizzare x, y, k e il nodo intermedio w . Per memorizzare l'indice di un nodo in un grafo da n nodi ho bisogno di $O(\log n)$ spazio.

Ma quante chiamate si “impilano” nello stack? La prima chiamata inizierà da $k = \log n$ ovvero misurare la lunghezza massima, poi ogni volta faremo $k - 1$ fino ad arrivare a 0, ogni livello ha sì due chiamate ma utilizzano sempre la stessa memoria quindi anche qui abbiamo $O(\log n)$. (Simile ad una ricerca binaria praticamente).

Mettendo tutto insieme abbiamo complessità di spazio:

$$O(\log n) \times O(\log n) = O(\log n)^2$$

Teorema di Savitch

Data una funzione $f(n) \geq \log n$ si ha $\text{NSPACE}(f(n)) \subseteq \text{DSPACE}(f^2(n))$

Il teorema ci dice che qualsiasi cosa noi facciamo in modo non deterministico utilizzando uno spazio $f(n)$ possiamo simularlo in modo deterministico utilizzando spazio $f(n)^2$.

Sia N una NTM t.c. $L(N) \in \text{NSPACE}(f(n))$. Per comodità facciamo in modo che N prima di accettare cancelli tutto il nastro si metta in uno specifico stato accettante q_{acc} . Chiamiamo questa configurazione c_{acc} mentre la configurazione di partenza q_{start} .

Adesso immaginiamo la macchina N non come un nastro ma un **grafo orientato**, il grafo $G_{N,w}$:

- Ad ogni nodo corrisponde una configurazione della macchina
- Gli archi collegano un nodo A ad un nodo B se le regole di N permettono di passare dalla configurazione A alla configurazione B durante l'esecuzione di w

A questo punto il problema non é più se la macchina N accetta ma se esiste un percorso che ci porta da c_{start} a c_{acc} .

Quanti nodi ha questo grafo? Per il teorema “Spazio limita il Tempo” abbiamo dimostrato che se una TM usa spazio $f(n)$ allora può assumere un numero massimo di configurazioni pari a $2^{O(f(n))}$, quindi il grafo ha un numero esponenziale di nodi, chiamiamo questo numero $m = 2^{O(f(n))}$.

Adesso possiamo definire una TM M che utilizza l'algoritmo PATH? chiedendo di trovare un percorso da c_{start} a c_{acc} lungo al massimo m passi.

Se M provasse a generare tutto il grafo occuperebbe spazio m , dobbiamo risparmiare spazio. L'algoritmo PATH? ha bisogno di sapere se esiste un arco tra il nodo A e il nodo B, M controlla le regole di base della macchina originale N e senza costruire il grafo risponde SI/NO.

- Sappiamo che PATH? su un grafo di m nodi consuma uno spazio pari a $\log^2 m$ e sappiamo che $m = 2^{O(f(n))}$
- Sostituendo otteniamo che la TM M occupa $O\left((\log(2^{f(n)}))^2\right)$
- Logaritmo e l'esponenziale base 2 si annullano quindi $O((O(f(n)))^2)$
- Otteniamo quindi $O(f^2(n))$

Corollari

- PSPACE = NSPACE: Se una macchina non-deterministica usa uno spazio polinomiale (es. n^3) allora per il Teorema di Savitch possiamo simularla deterministicamente usando il quadrato di quello spazio, quindi n^6 ma è sempre un polinomio.
- $NL \subseteq DSPACE(\log^2 n)$

Teorema

$$NL \subseteq P$$

Se $A \in NL$ allora esiste una NTM che decide A con spazio $O(\log n)$, questo significa che il numero di configurazioni è $2^{O(\log n)}$, per le regole dei logaritmi questo equivale ad un polinomio. Dato che il grafo delle configurazioni ha grandezza polinomiale ci basta usare PATH per risolvere in tempo polinomiale e quindi $NL \subseteq P$.

Definizione - Riduzione Logaritmica

$A \stackrel{l}{\leq}_m B$ se esiste $R : \Sigma^* \rightarrow \Sigma^*$ calcolabile in $O(\log n)$ spazio tale che $\forall x \in \Sigma^*, x \in A \Leftrightarrow R(x) \in B$

Questa ci serve appunto per mostrare la NL-Completezza, per mostrare infatti che tutti gli altri problemi sono riducibili ad un problema non possiamo usare una riduzione che impiega tempo polinomiale altrimenti questa potrebbe tranquillamente risolvere il problema senza tradurlo.

Però la difficoltà sta nel fatto che se una macchina usa spazio $O(\log n)$ può comunque girare per un tempo polinomiale prima di fermarsi, lavorando per questo tempo l'output finale potrebbe risultare troppo lungo per lo spazio, per scrivere l'output si usa quindi un altro nastro WRITE-ONCE che non conta ai fini della complessità dello spazio.

Definizione - NL-Completezza

B è NL-Completo se:

- $B \in NL$
- $\forall A \in NL, A \stackrel{l}{\leq}_m B$

Teorema

Se P, Q calcolabili in log-space anche $R(x) = Q(P(x))$ è calcolabile in log-space

Dimostrazione

1. R fa partire Q
2. Quando Q legge l' i -esimo bit del suo input e non lo trova

3. R mette in pausa Q
4. R fa partire P da 0, R ignora tutti i bit che sputa P ma tiene il conto con un contatore (che occuperá soltanto $\log n$)
5. Quando P restituirá l' i -esimo bit, R lo prende e lo passa a Q , spegne P e fa ripartire Q dalla pausa.

Corollari

- $A \leq_m^l B$ allora $B \in L \Rightarrow A \in L$
- $A \leq_m^l B$ allora $B \in \text{NL} \Rightarrow A \in \text{NL}$
- $A \leq_m^l B$ allora $B \leq_m^l C \Rightarrow A \leq_m^l C$

Teorema

PATH é NL-COMPLETO

Dimostrazione - Sappiamo che PATH é NL-HARD perché nel Teorema di Savitch abbiamo visto che una NTM M con spazio $O(\log n)$ accetta su $x \Leftrightarrow \exists G_{N,x}$ con $\text{PATH } c_{\text{start}} \rightsquigarrow c_{\text{acc}}$.

Ora dobbiamo mostrare che $\text{PATH} \in \text{NL}$ ovvero che può essere risolto da una macchina non-deterministica usando una memoria minuscola di $O(\log n)$.

Costruiamo quindi questa macchina:

- Su input G, s, t
- Se $s = t$ accetta
- Calcola in $O(\log n)$ spazio $n = |V|$
- Crea una variabile `currNode` e ci salva s
- Per $i = 1, \dots, n$
 - Sceglie in modo non-deterministico un nodo u
 - Se $u = t$ accetta
 - Se $(\text{currNode}, u) \in E$ imposta $\text{currNode} = u$
 - Se $(\text{currNode}, u) \notin E$ rifiuta.
- Rifiuta

La macchina non ricorda mai l'intero percorso ma soltanto tre cose:

- Il nodo finale t
- A che numero di passo é arrivata i
- Su quale nodo si trova in questo momento `currNode`

Quindi $3 \cdot O(\log n) = O(\log n)$.

Teorema - Time Hierarchy Theorem

Sia $t_1(n) \ll t_2(n)$ allora esiste $L \in \text{DTIME}(t_2(n))$ e $L \notin \text{DTIME}(t_1(n))$.

Il teorema ci dice che se abbiamo due classi di tempo con una grande differenza fra loro, allora esiste un problema che é risolvibile in quella più grande ma non in quella più piccola.

Dimostrazione - Per dimostrare che questo linguaggio esiste costruiamo la macchina $D(x)$:

1. Usiamo l'input x per costruire una TM M_x
2. Simula $M_x(x)$ per un tempo di $t_{1.5}(n)$ con $t_1 \ll t_{1.5} \ll t_2$
3. Se $M_x(x)$ accetta, rifiuto

4. Se $M_x(x)$ rifiuta, accetta
5. Se $M_x(x)$ non termina entro il timer, accetta.

Analisi:

- $L(D) \in \text{DTIME}(t_2(n))$ perché la macchina D impiega la maggior parte del tempo con il timer che è di soli $t_{1.5}$ e qualcosa di overhead. Quindi il tempo totale rimane $\leq t_2$.
- $L(D) \notin \text{DTIME}(t_1(n))$: Supponiamo ci sia un programma $Q \leq t_1(n)$ che è in grado di riconoscere lo stesso linguaggio di D , siccome il timer di D è $t_{1.5}(n)$ il timer è più che sufficiente. D riuscirà a finire la simulazione di Q .

Alla fine della simulazione D restituirà il contrario di Q . Se D dà sempre la risposta opposta a Q quando ricevo in input Q stesso, significa che D e Q stanno calcolando cose diverse, ma avevamo supposto che calcolassero lo stesso linguaggio. Il linguaggio di D è troppo difficile per stare in $\text{DTIME}(t_1(n))$.

Osservazione - Per fare in modo che la disuguaglianza asintotica funzioni dobbiamo aggiungere un valore junk di padding a Q fino a superare la soglia per cui $t_{1.5} \gg t_1$. In questo modo abbiamo n grandissimo e D ha tempo di finire l'esecuzione.

Teorema - Space Hierarchy Theorem

Siano $s_1(n) \geq \log n$ e $s_2(n)$ t.c. $s_2(n) \in \omega(s_1(n))$ allora $\exists L$ t.c. $L \in \text{SPACE}(s_2(n))$ ma $L \notin \text{SPACE}(s_1(n))$

Anche qui il teorema ci dice che se esistono due classi di spazio con una grande differenza fra loro, esiste un problema che posso risolvere in quella grande ma non in quella piccola.

Dimostrazione - Come prima costruiamo una macchina D che prende in input x e costruisce una macchina M_x :

- D segna sul suo nastro un'area di dimensione $s_2(|x|)$
- Inizializza un contatore su un altro nastro con un valore massimo $2^{O(s_2(n))}$, se il contatore supera questo limite significa che la macchina è andata in loop.
- Simula M_x :
 - Se M_x supera lo spazio allocato, accetta
 - Se il contatore raggiunge il massimo, accetta
 - Se M termina e accetta, rifiuta; se termina e rifiuta, accetta.

In questo modo otteniamo che:

- $L(D) \in \text{SPACE}(s_2(n))$ infatti abbiamo allocato dello spazio che la macchina non può superare.
- $L(D) \notin \text{SPACE}(s_1(n))$: Supponiamo esista un problema Q che riconosce lo stesso linguaggio di D usando spazio $s_1(n)$. Simuliamo Q con D , lo spazio allocato basterà e non andrà nemmeno in loop e quindi la simulazione arriverà fino alla fine ma la macchina restituirà il contrario, abbiamo una contraddizione.